

BRACE: Unified Benchmarking of Accuracy and Energy for Code Language Models

MOHAMMADJAVAD MEHDITABAR, Dalhousie University, Canada

SAURABHSINGH RAJPUT, Dalhousie University, Canada

ANTONIO MASTROPAOLO, William & Mary, USA

TUSHAR SHARMA, Dalhousie University, Canada

The rapid advancement of AI technologies and their accelerated adoption in software development necessitates a systematic evaluation of their environmental impact alongside functional correctness. While prior studies have examined sustainability in large language models, existing approaches lack systematic frameworks for evaluating accuracy-energy trade-offs in Code Language Models (CLMs). In this paper, we present a framework, BRACE, to benchmark CLMs on a unified scale of energy efficiency and functional correctness (referred to as accuracy). We benchmark 22 state-of-the-art models on code generation and summarization tasks, proposing two rating methods: Concentric Incremental Rating Circles (CIRC) and Observation to Expectation Rating (OTER). CIRC provides deterministic Euclidean-based rankings with static trade-offs that are robust to outliers, and OTER offers trend-aware evaluation with dynamic trade-offs that capture the complex correlation between energy and accuracy, each offering a distinct perspective and addressing the problem in a unique way. These rating methods enable us to rate LLMs on a 1-5 scale reflecting their combined capabilities in terms of energy efficiency and functional correctness. Our analysis reveals models generally perform better in the code summarization tasks as they are not enforced to generate a grammar-based and syntactically correct output. Also, we find that models' size does not have a significant impact on their ratings, indicating that if models utilize their parameters efficiently, they can be ranked higher on the energy-accuracy scale.

Additional Key Words and Phrases: Code Language Models, benchmarking, sustainability rating, energy consumption, green AI.

1 Introduction

Assigned-to ▶ tushar, saurabh ◀ Generative Artificial Intelligence (GenAI) tools powered by Large Language Models (LLMs) have achieved widespread adoption [14] due to their ability to generate human-like text, process complex queries, and assist users across various tasks [105]. In software engineering (SE), Large Language Models for Code (CLMs) such as Codex [22], which powers GitHub Copilot [37], are trained on vast corpora of source code and natural language to facilitate development activities. These models can generate, analyze, debug, and optimize code across multiple programming languages [30, 45, 52], transforming how developers approach programming tasks and increasingly being integrated into day-to-day workflows [69, 81, 112].

new ▶ CLMs are evolving rapidly, narrowing capability gaps with humans while expanding to new SE tasks [45, 102]. This evolution stems primarily from scaling laws—increasing model size to better capture the semantic space of SE tasks, thereby improving downstream accuracy [55]. However, this reliance on brute-force scaling introduces a trilemma of critical challenges. First, it carries severe environmental costs, as larger models consume drastically more energy [15, 74, 97] and emit more carbon [29, 58]. Second, it creates an operational bottleneck; a model may be highly accurate, but its massive memory footprint renders it infeasible for edge deployment or consumer-grade hardware. Third, it introduces a pervasive “Large Model Bias” into evaluation ecosystems. Comparing a 70B parameter model directly against a 7B parameter model obscures genuine architectural innovation behind raw physical scale. A massive model may achieve state-of-the-art accuracy but suffer from extreme structural bloat, utilizing its parameters highly inefficiently compared to a smaller, denser model.

Authors' Contact Information: Mohammadjavad Mehditabar, Dalhousie University, Halifax, Canada, javad@dal.ca; Saurabhsingh Rajput, Dalhousie University, Halifax, Canada, saurabh@dal.ca; Antonio Mastropaolo, William & Mary, Williamsburg, USA, amastropaolo@wm.edu; Tushar Sharma, Dalhousie University, Halifax, Canada, tushar@dal.ca.

Efforts to benchmark and mitigate these challenges have evolved, yet they consistently fail to evaluate these dimensions cohesively. A few research focused purely on sustainability. For example, the AI Energy Score [63] benchmarks LLMs solely based on their energy consumption, assigning a star-rating from 1 to 5. However, benchmarking LLMs only based on a sustainability metric, such as energy consumption, presents an incomplete perspective that neglects the fundamental role of functional performance and models size, which is key for practical adoption, as excluding them can be “show- stopper” in an accuracy-first or resource-constrained environment.

A few studies [7, 47] have applied Pareto optimality on accuracy and energy solely to identify non-dominated points — where no model outperforms another in both energy and accuracy. new While this approach does not consider the size of the models similar to past works limitations, it also suffers from three major limitations in the accuracy-energy modeling as well. ◀ First, it highlights only the dominant points, ignoring the rest. Second, without a mathematical function to combine the objectives, comparing points is unfeasible. Lastly, points on the frontier can differ significantly in overall performance, as models with extremely high value in one objective are not rewarded or penalized fairly. Similar to previous works pure energy-accuracy context, there has been some attempts to combine accuracy and energy aspects for benchmarking LLMs to assign a final score to models. In this direction, Kaplan *et al.*’s [54] attempts to jointly incorporate *F1-score* and carbon emissions by computing the ratio between normalized carbon emission and *F1-score*. However, ratio metric assumes a linear relationship between two objectives which overlooks the non-linear and complex interaction between two metrics. new Furthermore, Masciari *et al.* [68] proposed a linear approach of integrating carbon efficiency with models functional correctness which has the same limitation of pre-defined linear formula which is only guessed and not validated. ◀ Similarly, Jegham *et al.* [50] propose a scoring scheme built on top of DEA [21] that measures how effectively each model converts sustainability-related factors (*i.e.*, inputs of the technique) into multiple outputs represented by varying accuracy levels. Despite its novel conceptual framework, the approach suffers from overfitting [40], sensitivity to outliers [19], and other ranking issues [99].

new More holistically, a few approaches have attempted to incorporate physical size alongside energy and accuracy, but their process lacks interpretability and incorporation of fair trade-off. Specifically, Gjorgjevikj *et al.* [38] utilize PROMETHEE II, relying on linear preference aggregations that fail to capture non-linear covariance. Fischer *et al.* [32] propose a unified framework that discretizes metrics independently before merging them via a weighted median. This early discretization effectively erases continuous trade-off dynamics. Ultimately, forcing distinct physical and operational properties into a single, monolithic scalar is fundamentally opaque. A single score cannot intuitively explain whether a model was penalized for architectural bloat, excessive energy consumption, or poor functional accuracy. ◀

To fill this research gap, we present BRACE, the first comprehensive benchmarking framework for CLMs that systematically integrates functional correctness, energy consumption, and physical model scale without destroying interpretability. Rather than aggregating these competing dimensions into a monolithic scalar, BRACE decouples the evaluation into a robust, three pairwise profile (*i.e.*, energy-accuracy, energy-size, accuracy-size), mapping complex dynamics into an intuitive 1–5 Likert scale on 22 CLM.

First, to evaluate the empirical trade-off between energy and accuracy, we model it as a Multi-Criteria Decision Making (MCDM) [110] problem and introduce two rating methods to evaluate CLMs’ joint energy-accuracy performance. Specifically, our first rating method, Concentric Incremental Rating Circle (CIRC), is a distance-based approach that quantifies inefficiency of each model by the distance of that model to the most optimal achievable objectives.

We also propose Observation to Expectation Rating (OTER), a parametric model that captures the correlation between energy and accuracy, and evaluate each model’s efficiency by comparing observed and expected performance. Building on these methods, we assign rating between 1–5 to models, where the rating 5 denotes jointly strong accuracy and efficiency and 1 represents energy-hungry, low-performing models.

Second, to systematically eliminate Large Model Bias and account for deployment constraints, we integrate Neural Scaling Laws on accuracy-size and energy-size settings, to assess CLM architectural efficiency. By computing a trend-curve across the ecosystem, we evaluate how efficiently models convert their physical memory footprint into task accuracy (*Performance Efficiency*) and energy conservation (*Structural Efficiency*), each mapped into 1–5 rating. This explicitly rewards architectures and algorithmic process that achieve high utility through optimization rather than relying on structural bloat or extreme resource expenditure.

The five-point scale strikes a balance between sufficient granularity for model differentiation and practical interpretability, consistent with established energy rating frameworks [8, 32, 63].

While BRACE applies broadly to language models, we focus on *CLMs* as an ideal testbed for several reasons. First, coding tasks demand *strict correctness*: unlike natural language generation, where semantic variation is acceptable, code must compile, execute, and pass tests, enabling objective, deterministic evaluation. Second, *CLMs* require *cross-domain projection*: in code generation, models map natural language specifications to executable code, while in code summarization, they translate program semantics back to natural language explanations. This bidirectional translation between distinct formal and informal domains, rather than the *within-domain* generation typical of Natural Language Processing (NLP) tasks, creates unique computational demands that stress both the accuracy and efficiency of model architectures.

This study makes the following contributions.

- We propose two novel rating methods that jointly evaluate LLMs on functional correctness and energy efficiency to produce unified ratings.
- **new** ▶ We propose scaling-law-based Performance and Structural Efficiency metrics that account for deployment constraints and assess how effectively LLMs translate model size into accuracy and energy savings. ◀
- We present BRACE, a comprehensive benchmarking framework for *CLMs* on SE tasks that streamlines model evaluation and comparison, enables systematic model profiling, and establishes a foundation for future research in sustainable AI for software engineering.
- We publicly release BRACE [10], including code and data for replication and benchmarking new models.

2 Background

Assigned-to ▶ tushar, saurabh ◀ As explained in section 1, no suitable metric has been proposed that simultaneously accounts for efficiency, energy, and performance while providing a reliable overall score for all data points. **new** ▶ In this section, for simplicity, we consider only accuracy and energy consumption to illustrate the behavior of existing metrics under specific conditions and highlight scenarios in which they fail to capture desirable trade-offs. The same reasoning naturally extends to model size, which can be included as an additional objective or used in place of either accuracy or energy consumption. ◀

Before diving into the explanation of the metrics, we reformulate our problem to standard optimality problem, where we aim to maintain both aspect high without compromising each one of them. Also, in order to keep both aspects in the same bounded ratio and avoid unstable computation, we perform our operation on the normalized version of reported values. To this end, for energy consumption, a lower usage corresponds to a higher value. Hence, we will define the *energy efficiency* as ef_{norm} term to address this issue. We formulate this term as follows, where e represents a model's energy consumption on a task:

$$e_{norm} = \frac{e - e_{min}}{e_{max} - e_{min}} \quad (1)$$

$$ef_{norm} = 1 - e_{norm} \quad (2)$$

However, for accuracy, as we expect to achieve the highest accuracy, and they are already aligned in this direction, we only apply min-max scaling on accuracy values. The following equation shows how we derive acc_{norm} where acc represents a model's accuracy on a task:

$$acc_{norm} = \frac{acc - acc_{min}}{acc_{max} - acc_{min}} \quad (3)$$

Pareto Optimal

Pareto frontier is defined when multiple points can be considered as the most optimal points where none of the solution in pareto frontier set can be outperformed by any other one. In other words, improvement in one objective worsen the other objective [78]. In mathematic terms, we can say x^* is pareto-optimal if:

$$\nexists y \in \mathcal{X} : (A(y) \geq A(x^*) \wedge E(y) \geq E(x^*)) \wedge (A(y) > A(x^*) \vee E(y) > E(x^*)). \quad (4)$$

Where A is accuracy and E is energy efficiency in our problem. Pareto-Frontier face three major limitations in presenting a final score based on two objectives. Firstly, it can only determine the dominant points, leaving all other points unnoticed. Secondly, since it does not incorporate a mathematical function to integrate both aspect into a single score, it will make the comparison of each point unfeasible. Lastly, the points that lie in pareto front can be very different in terms of final performance, i.e., two points with significantly difference in one objective and relatively similar in the other objective can reside in the same pareto frontier. This will stem from the fact that pareto front only considers greater or lesser inequalities, no quantities and functional relations are involved. As a case study, we adopted ZEUS leaderboard energy and accuracy data [24], shown in Figure 1, to plot the pareto optimal models in their case. The pareto curve indicates four points as the dominant ones which indicate all of these would be considered as the most optimal models in same class. However, as a single example we observe both points a and b demonstrating almost same accuracy (b 's accuracy is slightly worse), while b 's energy efficiency is significantly higher than the a 's one.

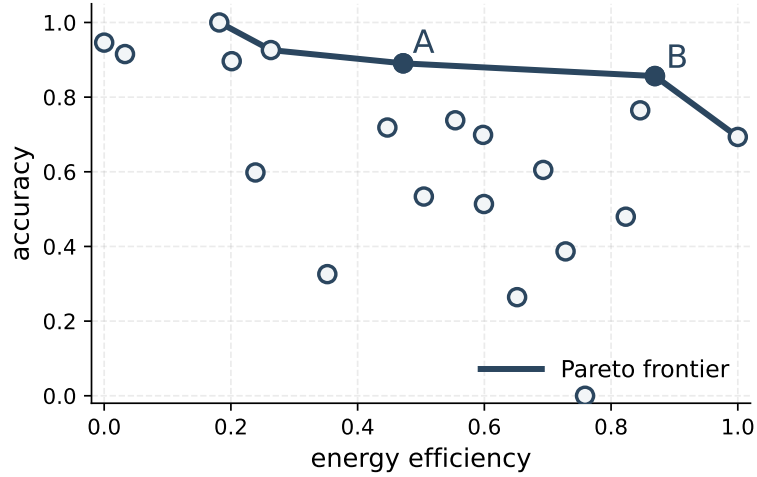


Fig. 1. Example of ZEUS data on two points a and b where it will assign a same label to them unfairly.

Mean Variants

The *arithmetic mean* is a straightforward operation commonly used when we want to maximize two equally weighted objectives simultaneously. By maximizing their sum, we define the final score as:

$$Score = \frac{eff_{norm} + acc_{norm}}{2}$$

Despite its simplicity and wide adoption, this metric fails to capture the true underlying relationship between the two aspects. Mathematically, the arithmetic mean assumes a strict 1:1 substitution rate between efficiency and accuracy (equivalent to a linear trade-off with a slope of -1). If the actual relationship follows a different slope or is non-linear, this approach becomes skewed. It will disproportionately favor improvements in one metric over the other, failing to assign logically consistent scores to points that represent equally valuable trade-offs.

An alternative interpretation frames the problem as a performance-to-cost ratio [23], treating accuracy as performance and energy consumption as cost. Using normalized values, the score becomes a simple ratio:

$$Score = \frac{acc_{norm}}{e_{norm}}$$

However, much like the arithmetic mean, this ratio metric fundamentally assumes a strict linear trade-off between accuracy and energy consumption—specifically, a proportional relationship passing through the origin. If the actual trade-off is non-linear, this approach breaks down. Because the ratio metric expects the cost-to-performance rate to remain constant, it fails to capture real-world dynamics, ultimately penalizing models in non-linear regions and evaluating trade-offs inconsistently.

Moreover, when we intend to prioritize one aspect over the other, arithmetic mean would turn into *weighted mean* of both metrics. We can describe this term as follows:

$$Score = W_1 \times eff_{norm} + W_2 \times acc_{norm}$$

However, this metric cannot capture relation between energy and energy efficiency beyond linear relation, and assigning a weight to each objective could be subjective and different in each task, leading to a inconsistent rating metrics across multiple scenarios.

Finally, *harmonic mean* and *geometric mean* are the last variants. Since the harmonic and geometric means show similar trends in aggregation, we focus more closely on the harmonic mean, which is also widely used in machine learning, with the F1-score being one of its most practical and widely used examples. We define the harmonic mean in our case as follows:

$$Score = \frac{2 \times eff_{norm} \times acc_{norm}}{eff_{norm} + acc_{norm}}$$

While it can score models by building a joint relation between two objectives, its scoring mechanism works regardless of objectives correlation and also penalizes low values of a single objective overly aggressive (harmonic mean is slightly more severe), which can undermine a strong performance in the other aspect.

To showcase an example of different mean-variant scoring methods, we present 2, where we assume a scenario in which accuracy and energy follow a polynomial relationship denoted by $y = (ax + b)^n$. For this relationship, we plot different scoring approaches. The left y axis represents the value of accuracy at each energy efficiency and the right y axis demonstrates how different mean approaches score. According to the figure, we observe first and last points in the plot, i.e., (0, 1) and (1, 0) are the most favorable to the arithmetic mean. However, in a fair and robust system these points should not be assigned with the highest score. On the other hands, harmonic mean tends to penalize low values in each objective, and tries to maximize scores when both objectives are closer to each other, having a reasonable distance from zero point. Therefore, we can observe where arithmetic mean shows it poorest performance, harmonic mean metric achieves their highest score relatively. Although harmonic mean tries to push both objective farther from

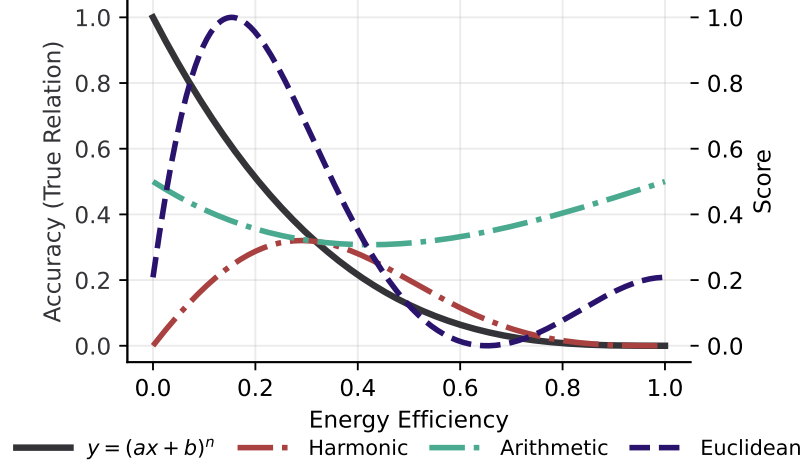


Fig. 2. Example of different mean variants behavior on a hypothetical relation between energy efficiency and accuracy

zero, its penalization is so strict, such that it does not allow a relative compensation for each objective. Hence, designing a fair and robust metric that could either consider the joint relation between energy and accuracy or score based on a reasonable trade-off, i.e., between arithmetic mean and harmonic mean, would be crucial to tackle this problem. So, our goal is to design a metric with the mentioned properties to provide a fair label for each model.

3 Methodology

Assigned-to ➤ tushar, saurabh◀

3.1 Problem Formulation

Our primary objective is to develop a comprehensive scoring framework that evaluates *CLMs* across three critical, often competing dimensions: *functional correctness* (accuracy), *operational cost* (energy consumption), and *deployment constraint* (model size).

new ➤ While prior approaches [32, 38] have attempted to merge multiple performance metrics into a single, monolithic scalar function—effectively modeling $s = f(A, E, S)$ —we argue that this mechanism fundamentally lacks interpretability. Aggregating all three dimensions simultaneously obscures pairwise trade-off dynamics and can mask critical failure points. For instance, a single score cannot intuitively explain whether a model was penalized for being an architectural bloated anomaly, overuse of energy, or low functional accuracy.◀

new ➤ **Problem Statement:** Let there be a set of models denoted as $M = \{m_1, m_2, \dots, m_k\}$, and a set of SE benchmarks represented as $B = \{b_1, b_2, \dots, b_{k'}\}$. Each model is executed on the selected benchmarks. From these runs, we collect the energy consumption profile of each model on three core devices, i.e., $D = \{cpu, gpu, ram\}$ along, with its corresponding accuracy score for a given benchmark. We obtain the energy consumption of a model on a benchmark as follows:

$$E_{m,b} \approx \sum_{d \in D} \sum_{n=1}^{N_{m,b}} P_{d,n}^{(m,b)} \Delta t, \quad \forall m \in M, b \in B \quad (5)$$

where Δt represents the sampling interval in seconds, $P_{d,n}^{(m,b)}$ denotes the instantaneous power consumption of a device in Watts, and $N_{m,b}$ is the total sampling count.

Let $A_{m,b} \in [0, 1]$ represent the raw accuracy score achieved by model m on benchmark b . Let $S_m \in \mathbb{R}^+$ represent the physical memory footprint defining the model's scale. For a given benchmark b , let $A_b = \{A_{m,b}\}_{m \in M}$ and $E_b = \{E_{m,b}\}_{m \in M}$ encapsulate the collective performance and energy distributions across the entire model ecosystem.

Our goal is to construct a multi-dimensional, discrete profile vector for each model-benchmark pair, denoted as:

$$R_{m,b} = \begin{bmatrix} R_{m,b}^1 \\ R_{m,b}^2 \\ R_{m,b}^3 \end{bmatrix} \in \{1, \dots, 5\}^3 \quad (6)$$

where each coordinate represents a Likert-scale score ranging from best (5) to worst (1). We define three distinct evaluation functions to map these raw metrics into our interpretable scale:

- (1) **Operational Energy-Accuracy Rating (R^1)**: Computed via a joint rating function φ_b that evaluates individual model trade-offs on energy and accuracy relative to the ecosystem's distribution and assign them a fair and robust rating:

$$R_{m,b}^1 = \varphi_b(A_{m,b}, E_{m,b} \mid A_b, E_b) \quad (7)$$

- (2) **Performance Efficiency Rating (R^2)**: Evaluated through a function, ψ_{acc} , that measures how efficiently a model converts its size into task performance by comparing its observed accuracy against the expected performance trend within the model ecosystem

$$R_{m,b}^2 = \psi_{\text{acc}}(A_{m,b}, S_m \mid A_b, S) \quad (8)$$

- (3) **Structural Efficiency Rating (R^3)**: Evaluated via a function ψ_{ene} that measures actual energy footprint optimizations against baseline architectural sizing trends:

$$R_{m,b}^3 = \tau_{\text{ene}}(E_{m,b}, S_m \mid E_b, S) \quad (9)$$

By isolating these ratings, the framework rewards highly optimized architectures that achieve top-tier performance without relying on extreme energy expenditure or structural parameter bloat, facilitating transparent model selection for software engineering contexts. ◀

3.2 Overview & Research Questions

This study aims to benchmark CLMs based on their combined operational efficiency (energy/size) and functional correctness (accuracy). To systematically explore this multidimensional problem, we articulate the following four Research Questions (RQs).

RQ1: How can we benchmark LLMs effectively based on their energy-accuracy trade-offs using a unified rating method?

Lasting solutions demand balance; prioritizing one goal in isolation risks undermining accuracy or amplifying environmental impact. Additionally, comparing models solely based on their raw energy-accuracy profiles is challenging, especially when trade-offs exist, *i.e.*, one model excels in accuracy while consuming significantly more energy, whereas another achieves better efficiency at the expense of precision. new ▶ Unlike parameter scale, which exhibits mathematically predictable behavior, the relationship between task accuracy and hardware energy consumption must be analyzed empirically and evaluated using a fair and robust scoring mechanism. ◀ Fairness ensures that moderately performing models are not

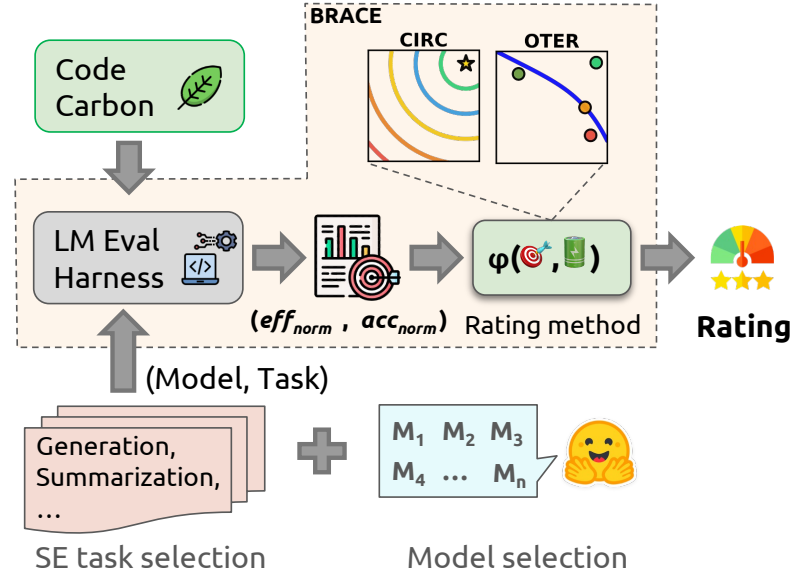


Fig. 3. Overview of the BRACE benchmarking framework. eff_{norm} and acc_{norm} are the normalized values of energy efficiency and accuracy of all models.

penalized, while robustness prevents over-rewarding models that achieve easy gains. The RQ explores approaches to unify energy-accuracy trade-offs.

new

RQ2: How do our proposed rating methods evolve and maintain stability as new models are added to the benchmark?

Evaluating ranking stability is crucial when introducing new Multi-Criteria Decision-Making (MCDM) methods for accuracy-energy objectives. Because benchmarking is an evolutionary process, a robust rating mechanism must scale seamlessly from a small initial validation set to any arbitrary size. It must remain interpretable, accurately reflecting new entrants while reliably adjusting existing models against emerging performance trends. Because our 1–5 scale derives ratings from the relative distributions of raw metrics (mapping the best and worst scores to 5 and 1, respectively), adding new models can inherently shift existing boundaries. This RQ systematically examines how these distributions evolve to determine which accuracy and energy profiles trigger minor versus significant rating shifts. Ultimately, this analysis validates whether our mechanisms are trustworthy. We investigate whether the benchmark preserves its primary trend during expansion or exhibits erratic behavior, ensuring that any observed rating shifts are mathematically justified, predictable, and meaningful. ◀

new

RQ3: How effectively can our proposed rating methods integrate dynamic weighting to capture user priorities in accuracy- or energy-first deployment environments?

While evaluating models with equal importance assigned to energy and accuracy serves as a robust baseline, real-world deployment contexts frequently impose asymmetric operational constraints. For instance, a mobile-edge coding assistant demands extreme energy efficiency, whereas a cloud-based, mission-critical code auditor prioritizes absolute precision. To accommodate these context-specific requirements, we extend our frameworks to support *priority elasticity* through weighted aggregation mechanisms. This RQ explores the integration of continuous weighting parameters (w_a , w_e) into the MCDM functions, allowing users to explicitly control the relative importance of accuracy and energy. We analyze how effectively these weights dynamically redefine trade-off penalties, tailoring the unified score to specific operational risk tolerances. Ultimately, this feature preserves the mathematical integrity and interpretability of the 1–5 ratings while significantly increasing the framework’s applicability across diverse, real-world engineering scenarios. ◀

new ▶

RQ4: To what extent do language models efficiently translate their physical parameter scale into predictive accuracy and energy efficiency?

While accuracy and energy consumption capture important aspects of model behavior, model size (i.e., parameter count and memory footprint) is also a critical factor. Size introduces a pervasive Large Model Bias into evaluation ecosystems, where direct comparisons between models of vastly different scales can obscure true architectural innovations behind sheer parameter count. A large model may achieve strong accuracy while utilizing its parameters inefficiently compared to a smaller, denser alternative.

Moreover, model size carries its own deployment cost. Even if a model is accurate and energy efficient, its memory and infrastructure requirements may make it impractical for consumer-grade hardware. In some cases, a model may simply be larger than necessary for the performance it delivers, suggesting that similar results could be achieved with fewer parameters and lower energy consumption. Therefore, size itself should be treated as a resource that models ought to utilize efficiently.

Accordingly, in this RQ, we investigate how efficiently different *CLM* translate their parameter budget into accuracy and energy efficiency. Similar to our first RQ, we derive ratings that enable direct comparison across models, which can later be combined with previous ratings to provide a more holistic evaluation. ◀

In this study, as outlined in Figure 3, we operationalize these RQs through BRACE, a comprehensive framework that evaluates *CLMs* along the critical dimensions of energy efficiency, task accuracy, and size-adjusted capability. The framework enables automated metric logging across predefined SE tasks, while our proposed scoring techniques project these complex multidimensional relationships into a unified, highly interpretable 1–5 rating scale, establishing sustainability and architectural density as primary criteria for modern LLM evaluation.

3.3 Benchmark dataset selection

This study aims to evaluate the capabilities of *CLM*. Therefore, our primary criterion for selecting benchmarks is their relevance to SE tasks. Given that the study benchmarks the sustainability characteristics of language models, we select two tasks to analyze model behavior across different problem types. To evaluate each model’s true capability and resource usage, we focus exclusively on generative tasks and adopt tasks that are widely used in SE research.

Our first task is *code generation*, which assesses a model’s ability to generate code that passes predefined test cases. We choose the livecodebench [49] benchmark, a recent and widely cited dataset that includes diverse code intelligence tasks [2]. We use pass@1 metric to assess models on this benchmark, which indicates whether the model generates a correct solution on its first attempt [22]. Our second task, *code summarization*, involves generating a docstring for a given code block to assess the model’s code comprehension ability [95]. We use the CodeXGLUE benchmark [61], a widely adopted and highly cited dataset [3, 4, 51]. We adopt its Python code summarization variant and use smoothed

blue [60] as the evaluation metric, following the original benchmark setup [61]. Both these tasks complement each other by capturing both text-to-code and code-to-text capabilities of the models.

3.4 Model selection

We use HuggingFace [28], the largest platform for open-source LLMs, for our model selection search. Given the SE focus, we only evaluate *CLM* in this study. We only consider open-source *CLM* so that can be deployed locally to measure their energy consumption accurately. Our initial search criterion selects models whose names include the term “code”, reducing the pool of models from 2, 105, 378 to 24, 840. We then filter for *text-generation* models, narrowing the list to 8, 840. Our computing environment (*i.e.*, 32GB GPU) can support models up to 9 billion parameters with full 32-bit precision. Therefore, we filter out the models with more than 9B parameters, resulting in 5, 607 candidates. From the set, we exclude all quantized variants to ensure consistency and fairness and select the top 25 base models by download count. new ▶ Following preliminary evaluation, models that required custom dependencies beyond the standard HuggingFace implementations, triggered Out-of-Memory (OOM) CUDA errors, or exhibited severe library incompatibilities were excluded, yielding a final cohort of 22 models. ◀ The list of the selected models and their download links can be found in our replication package [10].

3.5 Unified energy-accuracy scoring mechanisms

The problem studied in this paper is a classic Multi-Criteria Decision Making (MCDM) [110] with two objectives where we need to choose the optimal result from various alternatives pairs, *i.e.*, model energy-accuracy pairs.

To address this MCDM problem, we adopt the terminology and metrics defined in the background section 2. In fact, since our goal is to simultaneously maximize both objectives, we focus on acc_{norm} and $ef_{f_{norm}}$. We now introduce two novel rating methods that unify these normalized measures into a single scoring framework.

Concentric Incremental Rating Circle (CIRC). As discussed earlier, mean-based variants provide a potential way to jointly account for energy and accuracy. However, as outlined in section 2 and illustrated in Fig 2 different variants of the mean tend to be either overly lenient or excessively stringent. new ▶ In contrast, evaluating data points by their Euclidean distance (L_2) to the positive ideal point (1, 1) strikes a sound balance between the arithmetic and harmonic means. A key advantage of this approach is the L_2 squaring penalty. By squaring the performance deficits in X and Y prior to aggregation, the metric imposes an increasing marginal penalty on larger deviations. Consequently, a point cannot achieve a high rating merely by maximizing one attribute at the expense of the other; rather, minimizing the overall distance to perfection strictly requires balanced performance across all dimensions. ◀

Therefore, we propose **CIRC**, which builds upon Euclidean distance as its core foundation and leverages concepts from TOPSIS [48]— an MCDM algorithm to rank models by their Euclidean distance to the ideal most optimal and worst optimal solution. **CIRC** does not assume a correlation between energy and accuracy.

In the first step of **CIRC**, we define the optimal points for our setting. Unlike TOPSIS, which determines these points relatively, we leverage the normalized nature of our data to fix them at two absolute positions. Specifically, since both objectives are normalized within $0 \leq o_1, o_2 \leq 1$, we set the best point at (1, 1)—representing maximum accuracy and energy efficiency—and the worst point at (0, 0) which are $\sqrt{2}$ away from each other based on Euclidean distance.

To rate models on a 1-5 scale, we partition the interval $[0, \sqrt{2}]$ into five equal segments of length $\frac{\sqrt{2}}{5}$, which serve as score boundaries. Each boundary is defined as a circle to ensure all points on it are at the same distance from the ideal, using the Euclidean-like equation: $(x - c_x)^2 + (y - c_y)^2 = r^2$. In fact, while **CIRC** does not assume correlation between

energy and accuracy, due to the circular Euclidean formulation, scores remain invariant if energy and accuracy change circularly, reflecting *CIRC* expectation of their relative variation. This setup constructs five concentric circles centered at the ideal point (1, 1), with radii increasing in equal increments, starting from $\frac{\sqrt{2}}{5}$, and ending at $\sqrt{2}$.

After constructing five incremental circles, we assign the highest score to the model that lies within the first circle, *i.e.*, the circle nearest to the ideal point. Models with distances between the first and second radii receive a score of four, and this process continues until the region between the fourth and fifth circles, which is assigned a score of one. Hence, we define the entire process of *CIRC* rating as the following equation:

$$\begin{aligned} (x_i - 1)^2 + (y_i - 1)^2 &= d_i^2 \\ r_k &= k \frac{\sqrt{2}}{5}, \quad k \in \{0, 1, \dots, 5\} \\ R_i^1 &= 5 - k, \quad \text{s.t.} \quad r_k \leq d_i < r_{k+1} \end{aligned} \quad (10)$$

new ▶ Priority Elasticity: Weighted *CIRC* ◀

new ▶ The baseline *CIRC* mechanism assumes symmetric importance, contributing energy efficiency and accuracy equally to the final rating ($w_a = w_e = 0.5$). However, real-world deployment contexts frequently dictate strict, asymmetric priorities. In an energy-constrained environment, a practitioner may need to weight efficiency at 0.8 and accuracy at 0.2. In extreme bounds, a user may require evaluation strictly on a single axis, neutralizing the other entirely.

To satisfy these requirements, we introduce a weighted formulation of *CIRC*. We define w_a and w_e as the continuous priority weights for accuracy and energy efficiency, respectively, bounded such that $w_a + w_e = 1$. The weighted distance metric is formulated as:

$$d_i = \sqrt{2w_a(1 - y_i)^2 + 2w_e(1 - x_i)^2}, \quad \text{s.t.} \quad w_a + w_e = 1 \quad (11)$$

where the final Likert rating is assigned based on d_i as defined in the baseline methodology.

This formulation introduces strict geometric elasticity. When $w_a = w_e = 0.5$, the boundaries form perfect concentric circles. For instance, as the user shifts priority toward energy ($w_e > w_a$), the circles deform into vertical ellipses, increasing the marginal penalty for energy inefficiency while loosening the penalty for accuracy loss. In the extreme case where $w_e = 1$ and $w_a = 0$, the ellipses degenerate into parallel vertical lines along the x-axis (0, 0.2, 0.4, 0.6, 0.8, 1.0). Consequently, the impact of accuracy is mathematically neutralized, and models are binned uniformly based purely on their energy efficiency, perfectly aligning with single-objective user demands. ◀

Observation to Expectation Rating (OTER). We propose *OTER* to address the limitation of linearity assumed by existing unifying approaches, such as arithmetic mean and ratio-based methods. These approaches implicitly assume that an increase in energy consumption should correspond to a proportional, linear increase in accuracy to maintain balance. However, this assumption is fundamentally flawed, as the relationship between energy and accuracy can be nonlinear and vary across models [7, 53]. Improvements in accuracy do not necessarily require energy consumption to increase at the same rate [7, 24]; therefore, a method is needed to fairly and dynamically capture this compensation based on observed patterns.

To address previous limitations, we propose *OTER* method inspired by SFA [5]—a parametric method that models input–output relationships to define an optimal frontier—but enhanced with several novel integrations tailored to our problem.

To design *OTER*, we first postulate that as models become more energy efficient, a certain degree of accuracy compromise can be acceptable [15, 111], a pattern also observed in Pareto-front analyses [7, 47]. This assumption

enables us to (i) reward models that improve both objectives relative to weaker models and (ii) explicitly learn the energy–accuracy trade-off, rather than ignoring it as in prior approaches. We reformulate the problem in two dimensions, with energy efficiency on the x -axis and accuracy on the y -axis, and fit a monotonically decreasing reference curve capturing the dominant empirical trend in the data—unlike SFA, which estimates an optimal frontier of top performers. This curve provides an interpretable notion of the acceptable trade-off, indicating the expected accuracy at each efficiency level.

Based on this curve, models are rewarded or penalized depending on how their accuracy compares to the expected level, measured as the ratio of actual to expected accuracy. Specifically, models above the expectation curve demonstrate superior performance, as the general trend suggests a lower expected accuracy at their efficiency level, yet they exceed it. In contrast, models below the curve fail to reach the expected accuracy, indicating inefficiency in translating energy consumption into accuracy compared to the dominant behavior observed in other models. Finally, we rate models on a 1-5 scale by dividing the range between the minimum and maximum of the ratio derived from actual to expected accuracy into five equal intervals and assigning each model to the appropriate interval. We formulate this process using the following equation.

$$\begin{aligned} \mathcal{D} &= \{(x_i, y_i)\}_{i=1}^n, \\ \arg \min_{f \in \mathcal{F}} \sum_{i=1}^n \ell(y_i, f(x_i)) \quad &\text{s.t. } f(x) \text{ decreasing} \\ v_i &= \frac{y_i}{f(x_i)}, \quad v_{\min} = \min_{1 \leq i \leq n} v_i, \quad v_{\max} = \max_{1 \leq i \leq n} v_i, \\ \Delta &= \frac{v_{\max} - v_{\min}}{5}, \quad R_i^1 = \max(1, \lceil \frac{v_i - v_{\min}}{\Delta} \rceil) \end{aligned} \quad (12)$$

where $\mathcal{D}$ represents energy (x)–accuracy (y) pair for all considered models. f represents a monotonically decreasing function that best approximates these data points, by optimizing the objective function of ℓ . A model’s raw score, v_i , is defined by dividing the observation, *i.e.*, y , by the expectation, *i.e.*, $f(x)$. Δ represents the score interval for each class, and the final rating R is assigned based on which interval a model’s score falls into.

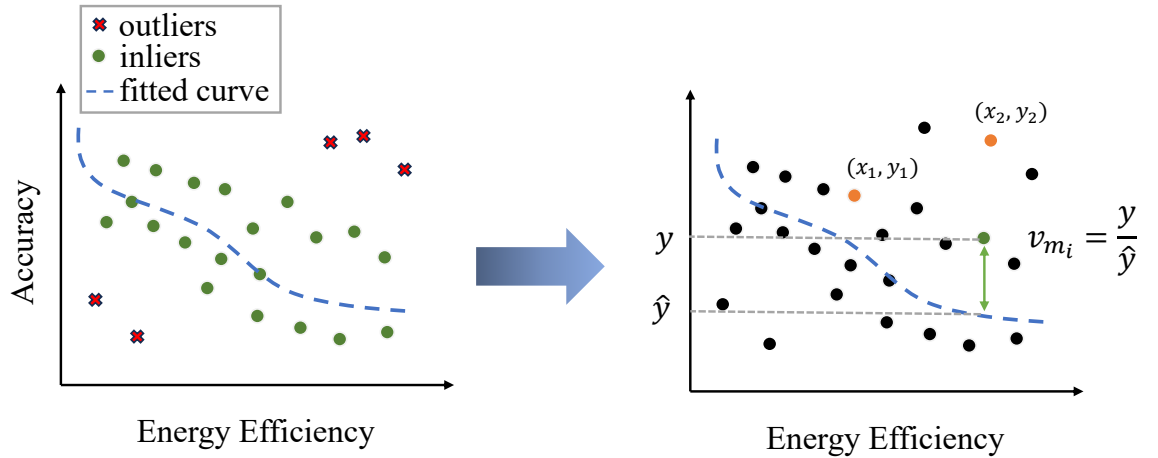


Fig. 4. Overview of the **OTER** approach. We first fit a curve to the majority of the points, followed by computing a raw score v as the ratio of a model’s observed to its expected accuracy.

Figure 4 illustrates an overview of the rating approach.

As discussed earlier, **OTER** favors models that maintain or improve accuracy as energy efficiency increases. This property ensures fairness, as a model outperforming another in both objectives should naturally receive a higher score. Such behavior is achieved through the monotonically decreasing nature of the fitted curve. An example of this can be seen in Figure 4 for two points (x_1, y_1) and (x_2, y_2) , which can be formally expressed as follows:

$$\begin{aligned}
 &f(x) \text{ decreasing, } V(x, y) = \frac{y}{f(x)}, \quad y_2 \geq y_1 \geq 0. \\
 &x_2 > x_1 \Rightarrow f(x_2) < f(x_1) \\
 &\Rightarrow \frac{1}{f(x_2)} > \frac{1}{f(x_1)} \quad (f(x) > 0) \\
 &\Rightarrow \frac{y_2}{f(x_2)} > \frac{y_1}{f(x_1)} \Rightarrow V(x_2, y_2) > V(x_1, y_1).
 \end{aligned} \tag{13}$$

For models whose accuracy decreases as energy efficiency increases, their raw score depends on the curve and their deviation from it. Points above the curve achieves raw score of more than 1, while those below score less than 1, indicating poorer performance. Models closer to the curve receive scores closer to 1, placing them near the center of the distribution. Whereas models that deviate most from expectation receive the highest or lowest scores, reflecting exceptionally strong or weak energy-accuracy trade-offs.

Like **CIRC**, **OTER** assumes no inherent energy-accuracy correlation and avoids static assumptions by dynamically estimating compensation from observed data, using a monotonically decreasing curve only to ensure fair reward allocation.

According to Equation 12, two components must be determined to complete the functionality of **OTER**: first, an optimization algorithm that minimizes the error term l while satisfying the constraint of decreasing monotonicity, and second, the function $f(x)$.

To perform the optimization step, we adopt quadratic programming with linear constraints, as it strictly enforces monotonicity—unlike genetic or learning-based methods—and can fit smooth functions, in contrast to isotonic regression, which yields step-wise solutions [41, 91]. Also, it is guaranteed that at least one decreasing solution exists regardless of its proximity to the actual data, enabling quadratic programming to minimize prediction error while controlling curvature and monotonicity through linear constraints.

We model $f(x)$ as a polynomial for three reasons. First, polynomials effectively approximate bounded regions ($0 \leq x, y \leq 1$) [13], a capability guaranteed by the Stone-Weierstrass theorem [107]. Second, they naturally encode monotonicity and convexity, facilitating the optimal solutions required in quadratic programming [57]. Finally, polynomials are widely established in optimization literature [12, 57, 94].

Subsequently, since the number of samples in our setting is limited and a few outliers may distort the fitted curve, we remove outliers (as shown in Figure 4) as an auxiliary step to ensure robustness. We employ Mahalanobis distances computed from a Minimum Covariance Determinant (MCD) [86, 87] which is highly robust to outliers [46] and correlation-aware [26]. In our settings, observations with squared robust distances exceeding the chi-square threshold at the 95th percentile with 2 degrees of freedom are marked as outliers.

Moreover, we need to determine the variables associated with the monotonic nature of the curve. As we intend to fit the curve in a decreasing way, we must ensure that for all points lying on the curve, their derivatives are always < 0 . However, when data shows atypical pattern, *i.e.*, higher energy efficiency corresponds to higher accuracy for majority of the point, the closest fitted function would be a near constant curve. The emergence of such a curve leads to unfair

scoring. In fact, if two models show similar accuracy but differ significantly in their energy efficiency, this curve fitting becomes unfair due to the scoring mechanism $y/f(x)$. Since their y values are similar and $f(x)$ is nearly constant, their raw scores will also be similar, even though one model is much more energy efficient and should receive a higher score.

Accordingly to address this issue, *i.e.*, ensure a fair and robust decreasing slope, we introduce the *Least Expected Slope (LES)*. Specifically, we compute pairwise slopes for all points that demonstrate an energy-accuracy trade-off, *i.e.*, demonstrating negative slopes. Consistent with systems benchmarking practices [9, 103], we use Q3 *i.e.*, the third quartile of computed slopes. Since the slopes are negative, this mechanism offers a conservative upper bound on the expected derivations. We configure the quadratic programming constraints to be less than *LES* in the entire range. We present *LES* formulation as below:

$$D := \left\{ d_{ij} := \frac{y_j - y_i}{x_j - x_i} : i \neq j, x_j \neq x_i, d_{ij} < 0 \right\} \quad (14)$$

$$LES := \text{Quantile}_{0.75}(MCD(D)), \quad LES < 0$$

Moreover, we aim to locate the curve always above zero, which can be ensured with a constraint at $f(x = 1)$, *i.e.*, minimum value, due to the monotonicity nature of the function. new ▶ Also, we should limit the maximum achievable accuracy to one. ◀ Consequently, to fit the most optimal function, satisfy the *LES* constraint, and ensure the curve remains above zero and below one, we formulate our quadratic programming problem as below:

$$\begin{aligned} & \min_{\beta \in \mathbb{R}^{p+1}} \sum_{i=1}^n \left(y_i - \sum_{k=0}^p \beta_k x_i^k \right)^2 \\ \text{s.t. } & \sum_{k=1}^p k \beta_k x^{k-1} \leq LES \quad \forall x \in [a, b]. \\ & \sum_{k=0}^p \beta_k \geq \epsilon \quad \text{s.t.} \quad \epsilon = 1e^{-2} \\ & \beta_0 \leq 1 \end{aligned} \quad (15)$$

In this equation, the first line defines the objective function that minimizes the error between predictions and ground-truth. The second line indicates the linear constraints required by quadratic programming. The constraints enforce that each derivative remains less than *LES*. The third constraint ensures the curve always lies above zero. new ▶ The fourth constraint limit the model accuracy to be at most 1. ◀ We use a polynomial degree of five to balance the flexibility-stability trade-off: higher degrees may cause instability [16], whereas lower degrees may not capture high variance.

new ▶ Priority Elasticity: Weighted *OTER*

Similar to weighted *CIRC* method, we extend *OTER* to support user-defined importance weights, denoted as w_a for accuracy and w_e for energy efficiency, subject to the constraint $w_a + w_e = 1$.

To systematically construct a weighted formulation, we first deconstruct the functional mechanics of the baseline raw score v_i :

$$v_i = \frac{y_i}{f(x_i)} = y_i \times \frac{1}{f(x_i)} \quad (16)$$

This decomposition reveals that the score is driven by two cooperative, monotonically increasing components. The first term, y_i , explicitly rewards higher absolute accuracy. The second term, $\frac{1}{f(x_i)}$, acts as a proxy for energy efficiency; since $f(x)$ is a monotonically decreasing curve, its reciprocal $\frac{1}{f(x)}$ scales positively with energy efficiency (x).

Because these terms interact multiplicatively, introducing scalar weights (e.g., $w_a \cdot y \times w_e \cdot f(x)^{-1}$) is mathematically invalid, as the scalars would merely merge into a single constant. Instead, we must apply weights as exponents, inspired by the Cobb-Douglas production function ($P = bL^k C^{1-k}$) [25] widely established in economic [65, 77] and time-cost [93] utility modeling.

An intuitive approach to introducing objective preferences would be to apply exponents directly to these isolated components, analogous to a standard Cobb-Douglas production function:

$$S_{\text{naive}} = y_i^a \times \left(\frac{1}{f(x_i)} \right)^b \quad (17)$$

While mathematically straightforward, this naive formulation introduces an asymmetric distortion at the boundaries of a discrete, interval-based rating system. If a user sets an absolute preference for accuracy ($a = 1, b = 0$), the score simplifies directly to $S = y_i$. Because accuracy scales linearly, dividing the score range into equal intervals yields a uniform, equitable distribution of Likert ratings. Conversely, if a user specifies an absolute preference for energy efficiency ($a = 0, b = 1$), the score becomes $S = \frac{1}{f(x_i)}$. Because $f(x)$ is a non-linear polynomial, equal intervals of its reciprocal do not map to equal intervals of the actual energy efficiency variable x . Consequently, the final ratings become highly skewed and compressed (e.g., placing models with vast efficiency differences into the same bucket), violating the core requirement for uniform rating distribution under targeted objective isolation.

To resolve this boundary non-linearity while preserving the elegant scaling of the Cobb-Douglas model, we propose a piecewise scoring function. Instead of treating the metrics as entirely decoupled base variables, we utilize the baseline ratio $\frac{y}{f(x)}$ as a unified model efficiency factor, and strategically introduce explicit linear parameters (y or x) depending on which domain of preference dominates. The weighted score function $S(x, y)$ is formulated as follows:

$$S(x, y) = \begin{cases} y^{2w_a-1} \left(\frac{y}{f(x)} \right)^{2w_e}, & \text{if } w_a > w_e \\ x^{2w_e-1} \left(\frac{y}{f(x)} \right)^{2w_a}, & \text{otherwise} \end{cases} \quad (18)$$

s.t. $w_a + w_e = 1, \quad w_a, w_e \in [0, 1]$

This piecewise architecture achieves a seamless, continuous transition between objectives while guaranteeing strict linearity at both boundary extremes through distinct algebraic mechanisms.

Case 1: Accuracy Dominance ($w_a > w_e$). When the user places greater emphasis on accuracy, w_a spans $(0.5, 1]$ while w_e spans $[0, 0.5)$. By substituting $w_a = 1 - w_e$, the first case of Equation 18 can be algebraically expanded and simplified:

$$\begin{aligned} S(x, y) &= y^{2(1-w_e)-1} \cdot y^{2w_e} \cdot \left(\frac{1}{f(x)} \right)^{2w_e} \\ &= y^{1-2w_e+2w_e} \cdot \left(\frac{1}{f(x)} \right)^{2w_e} \\ &= y^1 \times \left(\frac{1}{f(x)} \right)^{2w_e} \end{aligned} \quad (19)$$

This reduction demonstrates the underlying structural logic of the design. Throughout the entire accuracy-dominant domain, the absolute accuracy term y is maintained at a constant, linear exponent of 1. The user's preference is modulated entirely by the exponent $2w_e$ applied to the non-linear energy efficiency curve $\frac{1}{f(x)}$. As w_e decreases from 0.5 toward 0, the penalization/reward impact of the empirical curve is continuously attenuated. At the absolute boundary ($w_a = 1, w_e = 0$), the exponent collapses to 0, yielding $S(x, y) = y^1 \cdot 1 = y$. This preserves perfect linearity and ensures an equitable, uniform distribution of ratings based purely on accuracy.

Case 2: *Energy Efficiency Dominance* ($w_e \geq w_a$). When energy efficiency is prioritized, w_e spans $[0.5, 1]$ while w_a spans $[0, 0.5]$. Expanding the second case of Equation 18 yields:

$$S(x, y) = x^{2w_e-1} \cdot y^{2w_a} \cdot \left(\frac{1}{f(x)}\right)^{2w_a} \quad (20)$$

To understand the design logic here, we analyze how the energy efficiency objectives balance out. Both variables x and $\frac{1}{f(x)}$ move in the same positive direction (higher energy efficiency increases both terms). Notice that the sum of their exponents is strictly conserved across the entire domain:

$$(2w_e - 1) + 2w_a = 2(w_e + w_a) - 1 = 2(1) - 1 = 1 \quad (21)$$

Because their combined exponent intensity remains exactly 1, the overall mathematical representation of energy efficiency is almost kept constant in trend. However, as w_e scales up from 0.5 to 1 (and w_a decays from 0.5 to 0), the composition of this efficiency representation shifts smoothly. The formulation gradually injects pure linearity by increasing the power of x^{2w_e-1} while simultaneously fading out the non-linear polynomial effect via $\left(\frac{1}{f(x)}\right)^{2w_a}$. Concurrently, the absolute accuracy term y^{2w_a} is continuously suppressed.

At the absolute boundary ($w_e = 1, w_a = 0$), Equation 20 simplifies exactly to:

$$S(x, y) = x^{2(1)-1} \cdot y^{2(0)} \cdot \left(\frac{1}{f(x)}\right)^{2(0)} = x^1 \cdot 1 \cdot 1 = x \quad (22)$$

By collapsing entirely to the linear parameter x , OTER guarantees that when a user isolates energy efficiency, the resulting ratings are perfectly equidistributed across a linear scale (e.g., intervals of 0.0, 0.2, 0.4, ..., 1.0).

Finally, the continuity of the entire framework is preserved at the perfectly balanced midpoint ($w_a = w_e = 0.5$). Evaluating under Case 2 (where $w_e \geq w_a$ holds true) yields:

$$S(x, y) = x^{2(0.5)-1} \left(\frac{y}{f(x)}\right)^{2(0.5)} = x^0 \left(\frac{y}{f(x)}\right)^1 = \frac{y}{f(x)} \quad (23)$$

This maps precisely back to the baseline raw score v_i , confirming that the piecewise adjustments smoothly merge at the center while providing rigorous, linear interval protections at the extremes. ◀

new ▶

3.6 Size Efficiency of Models

To systematically mitigate “Large Model Bias” and isolate genuine architectural and algorithmic innovation from raw parameter scale, we build upon empirical Neural Scaling Laws [55]. These laws demonstrate that downstream performance and resource consumption [29] scale predictably with compute and model size. We also observe that models of similar size can exhibit substantial differences in accuracy and energy consumption due to architectural and implementation choices [7, 18, 47]. This motivates our investigation into how efficiently *CLM* translate their capabilities into performance while managing resource consumption.

To this end, we construct a single, cross-ecosystem baseline curve for both size-accuracy and size-energy trajectories instead of fitting separate scaling curves for individual model families. In fact, this approach is theoretically justified in accuracy-size settings by past works. Specifically, Maia *et al.* [66] and Ruan *et al.* [88] establish that diverse model architectures share a fundamental scaling trajectory, differing strictly in how efficiently they convert resources into capabilities. Furthermore, Owen *et al.* [79] adopted a single universal scaling law across diverse architectures, stating that positive deviations from the universal curve are typically the result of algorithmic or architectural advantages. The same logic applies to energy-size relationships. A model that breaks from the trend is either wasting resources or managing them optimally to lower power consumption.

Furthermore, using model size rather than compute (FLOPs) as the scaling factor is justified by Caballero *et al.* [17] and Kaplan *et al.* [55], which demonstrated that scaling laws remain structurally invariant across the scaling parameter, *i.e.*, training compute, model size, or tokens.

Thus, to operationalize the functions ψ_{acc} and τ_{ene} declared in Section 3, we draw on the concept of “Capability Density” defined by Xiao *et al.* [104], which quantifies the functional utility contained within each unit of a model’s parameter footprint. They evaluate models using the ratio $\frac{f^{-1}(A)}{S}$, where the numerator represents the expected scale derived from a fitted curve and the denominator represents the actual parameter count. Inspired by this paradigm and building on the concept of architectural deviation from the scaling curve [79], we construct size-efficiency ratings by comparing observed performance directly against size-predicted expectations derived via bounded ecosystem regressions. Let $\hat{A}(S_m)$ and $\hat{E}(S_m)$ denote the baseline expected task accuracy and baseline energy consumption for a model of parameter size S_m , respectively. We define the continuous variations for Performance Efficiency (R^2) and Structural Efficiency (R^3) as follows:

$$\begin{aligned} R_{m,b}^2 &= \mathcal{D} \left(\frac{A_{m,b}}{\hat{A}(S_m)} \right) \\ R_{m,b}^3 &= \mathcal{D} \left(\frac{E_{m,b}}{\hat{E}(S_m)} \right) \end{aligned} \quad \text{s.t.} \quad R_{m,b}^2, R_{m,b}^3 \in \{1, \dots, 5\} \quad (24)$$

where $\mathcal{D}$ represents a uniform discretization function mapping continuous efficiency ratios into a discrete 1–5 Likert scale based on empirical ecosystem boundaries. Also, we define model size (S_m) not by raw parameter counts, but by its operational *memory footprint in Bytes*. Measuring parameter scale via memory allocation accounts for differences in precision formats (e.g., FP32, FP16, FP8, or etc.). A 7B parameter model in FP32 occupies twice the physical capacity of its FP16 counterpart, allowing it to retain more information per parameter. Furthermore, memory footprint directly dictates deployment feasibility; from a practitioner’s perspective, hardware constraints such as GPU VRAM limits act as hard boundaries for model selection.

The explicit derivation of the expectation curves $\hat{A}(S_m)$, $\hat{E}(S_m)$, and their corresponding discretization boundaries are detailed below. 

Performance Efficiency Expectation Rating (Size vs. Accuracy). Although original scaling laws were formulated for training-time validation loss [55], they extend naturally to inference-time task performance because validation loss and accuracy are strongly correlated [27, 44]. Because downstream accuracy is strictly bounded between 0 and 1, the scaling function must follow an S-shaped (sigmoid) monotonic curve [6, 79, 104].

To satisfy these constraints, we employ the equation proposed by Krajewski *et al.* [56], treating the model size (S) as the compute factor:

$$-\log(Q_i) = \frac{A}{S_i^\alpha} \implies \hat{Q}(S_i) = \exp \left(-\frac{A}{S_i^\alpha} \right) \quad (25)$$

where Q_i is the downstream accuracy, S_i is the model size in GB, and A and α are trainable coefficients tuned to our specific dataset. We fit this curve across a dataset of evaluated models $\mathcal{D} = \{(S_i, Q_i)\}_{i=1}^n$ using Ordinary Least Squares on the double-logarithmic transformation of the data.

$$\arg \min_{A, \alpha} \sum_{i=1}^n (\log(-\log(Q_i)) - (\log(A) - \alpha \log(S_i)))^2 \quad (26)$$

Once the optimal baseline function $\hat{f}_{\text{acc}}(S_m)$ is resolved, similar to **OTER**, a model’s efficiency is determined by its vertical relative deviation from the curve. Once the optimal baseline function $\hat{f}_{\text{acc}}(S_i)$ is established, a model’s efficiency is determined by its vertical relative deviation from the curve.

This purely size-based formulation, rather than compute (FLOPs)-based modeling that also accounts for training token count, provides a clear diagnostic framework. Specifically, the fitted curve assumes a token-saturated regime, consistent with the core assumptions of foundational scaling laws [55]. Hence, for models falling below the baseline, token starvation is a distinct possibility alongside structural bloat or architectural inefficiencies. Conversely, models exceeding the baseline demonstrate performance that surpasses this token-saturated expectation, indicating advantages in architectural efficiency, high-quality data curation, or superior training configurations.

To map these deviations into our discrete 1–5 Performance Efficiency rating, we compute the Performance Efficiency ratio v_i and discretize it uniformly:

$$v_i = \frac{Q_i}{\hat{f}_{acc}(S_i)}, \quad v_{\min} = \min_{1 \leq i \leq n} v_i, \quad v_{\max} = \max_{1 \leq i \leq n} v_i$$

$$\Delta = \frac{v_{\max} - v_{\min}}{5}, \quad R_i^2 = \max\left(1, \left\lceil \frac{v_i - v_{\min}}{\Delta} \right\rceil\right) \quad (27)$$



Structural Efficiency Expectation Rating (Size vs. Energy). The intuition behind Structural Efficiency is similar: models should not consume more energy than the expected baseline for their size. Models that consume less energy than the trend dictates are rewarded. Energy consumption relative to model size has been described as linear [29, 39] or log-linear [1, 31]. To capture this trend generalized across varying hardware setups, we model the baseline energy expectation using a flexible power-law formulation:

$$E = \beta S_m^\gamma \quad (28)$$

where E represents the observed energy consumption, and β and γ are benchmark-dependent, learnable scaling parameters. When $\gamma = 1$, the model simplifies to a strictly linear relationship, preserving generalizability [29, 39].

We evaluate energy consumption as the total energy consumed by a model across the entire generation, rather than energy per token. This choice accounts for downstream generation efficiency: a large, capable model may solve a task using fewer tokens and less overall energy than a smaller model that struggles, requires a lengthy generation, and ultimately consumes more total energy. While the smaller model naturally uses less energy per token, analyzing the entire generation is preferred as it penalizes over-generation and captures true resource inefficiency.

Hence, we fit Equation 28 over the model ecosystem by minimizing the log-linearized residual sum of squares:

$$\arg \min_{\beta, \gamma} \sum_{i=1}^n (\log(E_i) - (\log(\beta) + \gamma \log(S_i)))^2 \quad (29)$$

The resulting curve, $\hat{f}_{\text{energy}}(S_m)$, yields the expected energy cost for an arbitrary model footprint. Because this size-energy modeling spans diverse architectures of varying sizes, the curve captures the macro-level industry trend. At any given size, individual models may still exhibit significantly different energy profiles. Because energy represents an operational cost, the optimization direction is inverted compared to accuracy. Models residing below the fitted trajectory consume fewer resources than expected for their size, suggesting they are either highly architecturally optimized or use fewer tokens than the baseline trend predicts. Conversely, models resting above the curve exhibit either extreme structural bloat or a tendency to over-generate tokens for that specific task. To maintain mathematical consistency with our unified scoring system, the Structural Efficiency rating (R^3) maps these deviations such that lower energy consumption yields a higher score:

$$u_i = \frac{E_i}{\hat{f}_{\text{energy}}(S_i)}, \quad u_{\min} = \min_{1 \leq i \leq n} u_i, \quad u_{\max} = \max_{1 \leq i \leq n} u_i$$

$$\Delta = \frac{u_{\max} - u_{\min}}{5}, \quad R_i^3 = 6 - \max\left(1, \left\lceil \frac{u_i - u_{\min}}{\Delta} \right\rceil\right) \quad (30)$$

The inversion term ($6 -$ term) ensures that $R_i^3 = 5$ corresponds to the most structurally efficient architecture. ◀

3.7 Experimental setup

Libraries: BRACE leverages two open-source components: LM Evaluation Harness [34] and CodeGreen [84]. LM Evaluation Harness is a widely-used benchmarking platform supporting applications like OpenLLM HuggingFace Leaderboard [33], providing over 180 LLM benchmarks.

CodeGreen is a low-overhead, hardware-agnostic energy profiling framework for precise, sub-millisecond software power measurement across diverse systems

Energy Measurement Setup and Execution Protocol: We extracted energy measurements across four stages: (i) model configuration; (ii) tokenizer initialization; (iii) model initialization; (iv) and instance inference. We selected four stages to align with common LLM power benchmarking practice that separates setup overheads from pure *CLM* involvement [75], and because this four-tier granularity yields a reproducible pipeline that generalizes across models without requiring intrusive, fine-grained phase profiling *e.g.*, prefill vs. decode.

We set CodeGreen’s sampling interval to one second to balance sampling frequency and noise [85], using *process* mode to capture only the current process’s energy. We validated CodeGreen’s CPU readings against *perf* and found them consistent, ensuring no other processes ran during experiments. To ensure consistency, we executed each model-benchmark combination three times. All models retained default configurations with fixed random seeds for reproducibility, were offloaded to GPU during initialization, and executed sequentially using identical scripts. No remote HuggingFace downloads occurred to maintain consistency.

Hardware: Our primary machine features an RTX 5000 Ada GPU (32GB VRAM) and AMD EPYC 9554P 64-Core processor (755 GB). For reproducibility, we repeated experiments on a secondary machine with Intel Xeon Gold 5317 CPU and RTX 3070 Ti GPU (8 GB VRAM).

Ground-Truth: Our problem is an MCDM problem which, by its nature, does not have a single solution. Therefore, we base our two approaches on a widely used MCDM method, TOPSIS [42, 67], which produces a single score from multiple features, and a well-known econometric efficiency technique, SFA [35, 43], which estimates optimal performance and inefficiencies. We extend these methods with new functionality to tailor them specifically to our problem setting, namely a fair and robust five-class rating of *CLM*.

Framework Extensibility and Usability: While this study evaluates 33 models on two SE tasks, BRACE is designed for broader applicability. Its architecture decouples evaluation from implementation, allowing users to test any *CLM* on any benchmark simply by updating identifiers in the configuration file [10]. This enables practitioners to benchmark new releases, evaluate domain-specific SE tasks, or extend the framework to additional code benchmarks with minimal setup overhead and no code modification. Moreover, although we adopt the conventional 1–5 rating scale, the framework supports configurable rating ranges (*e.g.*, 1–10) to accommodate different granularity requirements.

4 Results

4.1 RQ1: Energy-Accuracy Rating

In this section, we present the evaluation results of all models and compare their performance on the unified scale of energy-accuracy using our rating methods.

Comparative Analysis on Two Benchmarks: Table 1 presents accuracy and energy efficiency reported for all the considered models for both the tasks. The table also shows the rating assigned by *CIRC* and *OTER* for all the models, along with their IDs. Seed-Coder-8B-Instruct (M_{18}) achieves a perfect score on LiveCodeBench and a moderate one on CodeXGLUE, making it the most optimal overall. This model, built on the Llama backbone, efficiently converts computational capacity into accuracy. Following it are the Qwen models (M_5 and M_7), attaining moderately high ratings on both tasks. They are the only models that do not sacrifice one objective for the other, maintaining a sound trade-off. In contrast, the largest Qwen model (M_6) performs worse than smaller variants on LiveCodeBench, highlighting that larger models do not guarantee higher accuracy. The weakest model, Yi-Coder-9B (M_{13}), shows the lowest energy efficiency and moderate accuracy, indicating inefficient parameter utilization. To validate that ratings reflect genuine

ID	Model	LCB		CXG		LCB Rate		CXG Rate	
		A	E	A	E	CIRC	OTER	CIRC	OTER
M ₁	deepseek-coder-6.7b-base	0.23	0.31	0.47	0.23	2	1	2	3
M ₂	starcoderbase-1b	0	0.93	0	0.88	2	1	2	1
M ₃	starcoder2-3b	0.02	0.23	0.37	0.32	1	1	2	3
M ₄	CodeLlama-7b-Instruct-hf	0.21	0.82	0.76	0.23	3	1	3	4
M ₅	Qwen2.5-Coder-3B-Instruct	0.52	0.95	0.36	0.57	4	3	3	3
M ₆	Qwen2.5-Coder-7B-Instruct	0.38	0.78	0.36	0.76	3	2	3	3
M ₇	Qwen2.5-Coder-1.5B	0.48	0.99	0.31	0.88	4	3	3	3
M ₈	deepseek-coder-1.3b-base	0.1	0.76	0.7	0.79	2	1	4	5
M ₉	deepseek-coder-7b-instruct-v1.5	0.58	0.77	0.35	0.35	4	3	2	2
M ₁₀	starcoder2-7b	0.04	0.15	0.56	0.21	1	1	2	3
M ₁₁	codegen-350M-mono	0.02	0.83	0.09	0.83	2	1	2	1
M ₁₂	Qwen2.5-Coder-0.5B	0.27	0.99	0.37	0.88	3	2	3	3
M ₁₃	Yi-Coder-9B	0.33	0	0.16	0	1	1	1	1
M ₁₄	rombos_Replete-Coder-Llama3-8B	0.21	0.74	0.2	0.28	3	1	2	2
M ₁₅	speechless-code-mistral-7b-v1.0	0.31	0.85	0.37	0.6	3	2	3	3
M ₁₆	stable-code-3b	0.06	0.68	0.53	0.67	2	1	3	4
M ₁₇	CodeLlama-7b-Python-hf	0.15	0.73	0.38	0.23	2	1	2	2
M ₁₈	Seed-Coder-8B-Instruct	1	0.88	0.31	0.59	5	5	3	2
M ₁₉	CodeQwen1.5-7B-Chat	0.06	0.99	0.03	1	2	1	2	1
M ₂₀	Magocoder-S-DS-6.7B	0.42	0.62	0.39	0.42	3	2	3	3
M ₂₁	granite-8b-code-base-4k	0.1	1	1	0.14	2	1	2	5
M ₂₂	codegen-2B-mono	0.02	0.64	0.45	0.51	2	1	3	3

Table 1. Summary of model performance on LiveCodeBench (LCB) and CodeXGLUE (CXG): normalized accuracy (A), normalized efficiency (E) and unified ratings (CIRC and OTER) for each benchmark.

trade-offs rather than systematic size bias, we conduct Kruskal-Wallis tests—a non-parametric method suitable for small samples without assuming normality [76, 92]. We group models by size ($< 3B$: $n = 6$, $3-7B$: $n = 5$, $\geq 7B$: $n = 11$) and test the null hypothesis that groups share the same rating distribution. Results fail to reject the null hypothesis ($p > 0.05$ for all: LCB-**CIRC** $p = 0.92$, LCB-**OTER** $p = 0.97$, CXG-**CIRC** $p = 0.30$, CXG-**OTER** $p = 0.40$), confirming no significant size-based bias.

Rating Methods Behavior Interpretation: We interpret how each rating method rates models based on their observed performance, shown in Figure 5. In the top-left plot, we observe **OTER** fits a curve to estimate the trend. It ranks over half the models in the weakest class. This results from the highly capable model M_{18} , which achieves perfect accuracy and near-optimal energy efficiency. Despite the presence of outliers and models exhibiting high energy efficiency but low accuracy, **OTER** does not favor them, demonstrating robustness.

The right-top plot demonstrates **OTER** rates the majority of models to middle classes as they reflect a balanced trade-off between energy efficiency and accuracy. The representation shows robustness at the highest energy efficiency, where moderate accuracy is expected, unlike M_2 , M_{11} , and M_{19} . It also fairly rewards the most accurate model with relatively low energy efficiency (M_{21}), assigning it the highest score as it is considered as an exceptional outlier.

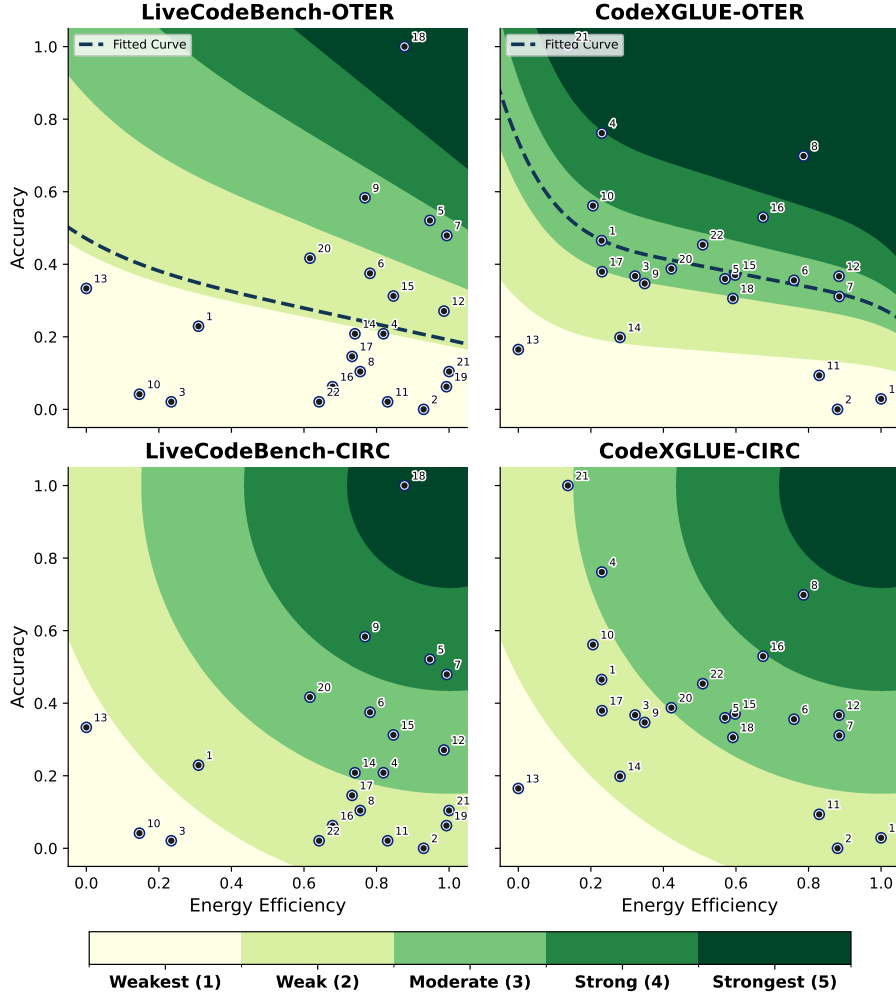


Fig. 5. Unified energy-accuracy model rating; numbers in the plot show models' IDs

To statistically validate *OTER*'s robustness to design choices, we conduct systematic sensitivity analyses on two critical aspects: hyperparameter selection and outlier removal. *OTER*'s methodology involves explicit design choices—polynomial degree (5), MCD outlier threshold (95th percentile), LES quantile (75th), and ϵ (0.01) at $f(x=1)$ —that could potentially introduce fragility. We test 358 hyperparameter configurations varying polynomial degree $\in \{3, 4, 5, 6, 7\}$, MCD percentile $\in \{90, 95, 97.5\}$, LES quantile $\in \{65, 70, 75, 80\}$, and $\epsilon \in \{0.001, 0.01, 0.1\}$ computing Spearman correlations between each configuration's ratings and our baseline. Results demonstrate exceptional stability: mean $\rho = 0.982$ (min $\rho = 0.922$), with all 358 configurations achieving $\rho > 0.85$ and maintaining 96.1% average rating stability (min 81.8%). All correlations are statistically significant ($p < 0.001$). To assess the critical role of outlier removal, we compare ratings with and without MCD-based coordinate filtering. We found that filtering is mathematically essential: without filtering, the regression curve is severely distorted by high-leverage points, dropping the Spearman

correlation to $\rho = 0.628$ on the CXG dataset and reducing overall average stability to 72.7%. This confirms that MCD-based outlier removal acts as a necessary stabilization mechanism, dramatically enhancing robustness to architectural breakthroughs without introducing systematic size bias.

The bottom left plot demonstrates the *CIRC* approach, which rates models regarding the circle that they fall into. Since this is a Euclidean-based ranking, both objectives are considered equally. This plot is predominantly dense in the weak class, as most models perform well on only one objective. The absolute distances along each curve allow fair comparison among models within the class. The bottom right plot also exhibits similar behavior. As a static scoring approach, *CIRC* leaves the very strong class empty, highlighting room for model improvement.

Having validated *OTER*'s hyperparameter robustness and confirmed size-agnostic fairness for both methods, we further assess rating stability and fairness under two practical deployment scenarios: whether individual models disproportionately influence the overall ratings, and whether measurement noise in energy-accuracy inputs destabilizes rankings. We conduct two targeted tests addressing these concerns. First, leave-one-out (LOO) analysis—removing each model sequentially—assesses dependency on individual data points. Results demonstrate high stability: *CIRC* achieves mean rating drift of 0.024 (LCB) and 0.022 (CXG) with Kendall- $\tau > 0.98$, while *OTER* maintains 0.037 (LCB) and 0.067 (CXG) with Kendall- $\tau > 0.96$, confirming that ratings remain consistent regardless of which model is excluded. Second, we test robustness to measurement noise by perturbing inputs with uniform noise ($\epsilon = \pm 5\%$) across 20 trials. Both methods are resilient: *CIRC* shows mean rating changes of 0.032 (LCB) and 0.059 (CXG) with worst-case shifts of one rating class, and *OTER* 0.068 (LCB) and 0.209 (CXG), with worst-case shifts of one and two rating class respectively. These results confirm both methods yield stable ratings robust to model influence and measurement uncertainty.

Difference in *OTER* and *CIRC* on LiveCodeBench:

In this section, we compare how *OTER* behaves differently from *CIRC*. We can observe in Table 1 and the first column of Figure 5 that the order of scores in both approaches are similar, with most models differing by only one class. In fact, *CIRC* is more lenient in scoring than *OTER*, assigning higher ranks to models that demonstrate at least one high objective. Nonetheless, we can conclude that both methods rank fairly with equal orders, only varying in the middle classes. It is also worth mentioning that due to the discretizing effect, we can observe many models that are close to boundaries, which a minor change in one of the values would cause their class to change. This fact can also lead to a slight variance in both of our proposed methods.

Difference in *OTER* and *CIRC* on CodeXGLUE:

For the CodeXGLUE benchmark, we observe more conflicts by analyzing the CXG rate in Table 1. This behavior arises from the relations that are captured by *OTER*. For instance, analyzing the right column of Figure 5 granite-8b-code-base-4k, i.e., M_{21} , is the model that is rated as “Strongest” in *OTER*, emphasize the severity of achieving high accuracy, while it is ranked as “Weak” in the *CIRC* approach, indicating its low energy efficiency. Furthermore, similar to *OTER* in LiveCodeBench, *OTER* in CodeXGLUE does not favor the highest energy-efficient models. This occurs because most models cluster within moderate accuracy (0.3–0.6) while spanning the entire energy efficiency spectrum, revealing that high efficiency is easier to achieve than high accuracy, as few models attain high accuracy regardless of their efficiency level. Overall, both methods are performing closely to each other with some exceptions that highlight their distinct behaviors depending on the perspective taken.

Benchmark Effects on Ratings:

In the first row of Figure 5, showing *OTER* results, scores are generally lower on LiveCodeBench. This reflects the higher complexity of generating structured, grammar-based text that also passes unit tests, compared to open-ended

text generation with a less rigid evaluation metric. Our novel ranking method captures this subtle point, with **OTER** remains trend-normalized per task and assigning higher expected accuracy to CodeXGLUE without trivially favoring the easier task. The second row in the Figure 5 shows model ratings using **CIRC**. While models on CodeXGLUE have higher average accuracy, reflecting the task’s relative ease, their low energy efficiency places most models in the second and third classes.

Moreover, variations in model ratings can be analyzed by comparing their scores under the LCB and CXG rates in Table 1. In general, variances are more pronounced in the **OTER** approach, where the fitted curve produces model-specific scoring. We highlight a few largest differences. The first model demonstrating the largest variance, deepseek-coder-1.3b-base (M_8), performs well on CodeXGLUE, achieving 0.7 accuracy and 0.81 energy efficiency, leading to scores of 4 and 5. However, on LiveCodeBench, its accuracy drops to 0.1 despite a reasonable energy efficiency of 0.78, achieving scores of 2 (**CIRC**) and 1 (**OTER**). This behavior is expected because **OTER** interprets high energy efficiency is relatively easier to achieve than high accuracy. Therefore, being a relatively small, the model struggles on the more demanding LiveCodeBench and leads to a large difference in scores across the two tasks. Another example of large difference in the scores is Seed-Coder-8B-Instruct (M_{18}). This model shows high energy efficiency on both LiveCodeBench and CodeXGLUE. However, the primary cause of its score difference is accuracy. It scores a perfect 5 on LiveCodeBench, but only 3 (**CIRC**) and 2 (**OTER**) on CodeXGLUE. This suggests that the model excels at code generation and completion but is less capable in code comprehension tasks.

Summary

- Models generally scored better in CodeXGLUE because of the easy nature of the task
- Since model accuracy shows low variance across most data points, **OTER** interprets high accuracy as harder to achieve and therefore rewards it more.
- **CIRC** showed as a reliable and deterministic algorithm, while **OTER** showed as a trend-aware and sensitive to outliers, which makes it robust.
- Trade-offs in **CIRC** are static while trade-off in **OTER** depends on the position of the point
- Models final scores can largely vary between two tasks in some cases, which is dependent on architectures and the pretraining phase, specifically how much emphasis was placed on code comprehension versus generation.

4.2 RQ2: Rating Distribution Shift

new ▶ As explained in the goal of RQ2, we aim to observe the impact of adding new models, what happens to previous ratings, justify the shifts, and whether the new set added are reliably fitting into the benchmark and deserve what they actually showing. So before diving into each method-benchmark justification, we should first explain that why rating shift is expected. So both methods use Likert-based scaling and min-max normalization, so these factors introduce relativity to the existing set, which is expected in bounded rating system. so in Likert-scaling it’s clear that most performant models maybe in current benchmark achieves the highest rating and the same intuition goes for the worst models, but obviously most performant models are not going to remain the same, as industry is developing and we see the trend that model developers are trying to make their models efficient and at the same time with the same capability or higher capabilities than existing set. other than that, it’s completely possible that the set of the models that we added are not comprehensive and there might exist some models which are too good or too bad which after bringing these models to our set, it’s expected that this new exceptional model for example gets the highest rating, *i.e.*, 5 and push the earlier models which were 5 to some lower ratings. so regarding the min-max normalization that both model apply that as a preprocessing

step, so min-max is standard normalization which is widely applied in ML in order to better process the models, and it's also widely recommended for MCDM methods to help achieve fair scores [70, 101]. and here again min-max helps shape *CIRC* in 5 concentric static formula which ensures that values are capped 1, and it's also suggested in TOPSIS [20, 72], which *CIRC* is inspired based on that, as well that the values should be normalized, and we apply min-max for *OTER* to help scale both objectives in the same bracket so that it makes it more interpretable and helps *OTER* fit and optimize its curve smoother and faster. so how normalization can change the values, so as already explained the min-max of our current set obviously is not representative of the all the models existing, there might come some model that is much different in either its accuracy or energy, so we have to take into account of that, and maybe the old min-max is not the current min-max after adding these new models, so when min-max is changed as a result the value of normalized acc and energy efficiency will change, so this will shift the model from the place where they initially were, so for both methods of *CIRC* and *OTER* their classes/rating changes as well specifically for the models that were previously on the boundaries. so overall saying that normalization is a relative formula and it must take into account of other models otherwise is not possible, and without normalization because of the reason said above our rating methods won't be fair, robust, smooth, fast and complied with best practices. also *OTER* has one other feature on top of the existing thing which is trend-aware, so think about if our selected set is not good and representative of all models or maybe the new models that are selected are completely different, so the trend will change, maybe those were previously highly efficient now they are not anymore because the new trend of efficient models can adapt curve so that for example the older models mostly go much below the curve which represent their incapability into converting the inefficiency to accuracy, so pushing the olders to much more inefficient and keep themselves as efficient, as a result, the scores for those ones should also change to reflect the true capability of the models, although we can use the same fitted curve and just place the new models in the same box and get the scores for the new ones without changing the scores for the old ones, we recommend refitting to be more fair to new models or even the old models because new trend means new set of capability which models should be able to unlock, existing with old expectation will be obsolete soon. and we do the same exactly here (refitting) so it dynamically refits its curve and captures the majority and the new trend, so it may assign completely new rating to old ones, so they might have not deserved those which already got. we should say that these are features that helps propose our robust and fair methods, and not something that is actually the problem of our approach. So, we have to also mention that while 1-5 rating makes it so much easier to pick your desirable models but it might makes you confused for example adding a new much good or bad model might shift most of the models towards the same class and higher, so to overcome this either increasing the rating classes make model more differntiable or using the raw score v_i that shows the absolute and the magnitude of the performance of the models make apple to apple comparison of the models easier if they lie within the same class. Our purpose was to show a big picture of the models performance, whether they are generally good and how can we apply 1-5 energy rating like appliance of kitchen that consider performance as well and show a grand-scheme of our ratings and where do LLMs currently stand in terms of performance and efficiency. so let's first see where the new models are mostly positioned in terms of their accuracy and energy efficiency in comparison with the old models for LCB benchmark we see that our previous set min-max remained intact as the new models are mostly leaning towards low to moderate level of accuracy while showing themselves as highly energy efficient, so overall not changing min-max and falling somewhere in the middle. However, for CXG (codexglue) the min-max is changed actually M_{33} which is prism-coder-7b, so this model only produced one token per sample, probably not understanding the template and the prompt unlike other models, so it just finished in 26 seconds as a result it couldn't satisfy the task requirement to generate a understandable and aligned code summary docstring achieving lowest accuracy but at the same time the highest energy energy efficiency. so this casue a change in min-max normalization and as a reuslt shifting models a bit, for example M_2 which previously performed the worst in terms of accuracy now got shifted 0.05 and M_{19} which was the most energy efficient one now got decreased by 0.05 ending in 0.95 even though all the reasons above said that good rating method is actually a rating method that adapts to new set, we still expect to see almost similar trend in the score of old models as some stability is expected in our methods and our set actually quite have wide

range of the models, unless we see a strange pattern when new models are added so in order to see how much changed is observed for the first 22 models before and after adding these new models we apply kendal tau test to see if they mostly align, so kendal tau showing perfect 1 for *CIRC* and 0.84 and 0.93 for *OTER* livecodebench and codexglue benchmark respectively. showing that our methods stayed almost firm without deviating too much from their first score, confirming the robustness of our methods. this makes sense because the trend of the new models added are mostly similar to the old models showing in table 2 and figure 6. so lets analyze *OTER* first, on the LCB benchmark on the figure 6, let's see first what happened to the fitted curve and as a result, 1-5 regions. so since new models are mostly dense within right bottom part of the figure, indicating highly energy efficient and low accuracy, so already in the previous RQ we said that mostly models are dense in this area, so adding new models will resonate/intensify its impact, so the curve will see that achieving high efficiency is not hard for the models instead achieving high acc is difficult for the models, so the curve should pass through the average of these points, and what differed now is that so for the previous case where we had 22 models for example at $f(x = 0)$, indicating zero energy efficiency, the curve expected the model show average accuracy, but here as achieving energy efficiency is emphasized and resonated which is easier, the curve saying that at zero energy efficiency model should achieve the best accuracy, because there are only scarce models showing low energy efficiency. this behavior led to a larger region of 1 rating class for the models showing low energy efficiency, and also caused that we have a more steep curve than the previous one (in fact, steep curve is because of the constraint we set that the curve should not exceed one accuracy, and also those models showing low energy efficiency are considered as outliers and the curve cares less about them and mostly see that models are filled with energy efficiency 0.5 to 1 with acc spanning from 0 to 1, that's why the curve is more steep). so why we see different shape of regions rating classes than the previous ones and whether it's fair. so new models after coming again showed highly energy efficient and very poor on accuracy so this increased the majority and pulled the curve down in fact convinced the curve that at highly energy efficient, compromise on accuracy can be more, so earlier in figure 5 we saw more strictness in a sense that at maximum energy efficency the curve showed that it expect to see higher accuracy than we currently see in the figure 6 taking the level of hardness to achieve a good acc on highest energy efficiency spectrum. and the reason we see more steepness in the 1-5 region is exactly again as expected the curve is more steep so at earliest spectrum the expectation is so high from the models, for example even showing highest accuracy with lowest energy efficiency will get the lowest rating, as we see most models achieved high energy efficency, but as we go on and the trend show that highly energy efficnet ones are not capable of achiving good accuracy the regions says that now that they are at least showing good energy efficiency we can take their behavior into account and award them more, so saying that if you are showing good energy efficiency the level of expectation of accuracy is now lower we reward you to get better ratings. so overall the trend we see that here at earlier energy efficiency is much more strict and pushed the higher rating classes higher because the majority showing highly energy efficient, so achieving eff must not be hard for the models, so those who are showing low eff must be penalized more, but as we go on to high energy efficiency model is more lenient because majority showing much lower acc than previous one, so pulled the rating classes a bit down to take into account of that. for example we can see rating class of 4 has shifted right because earlier at lower energy efficiency it didn't penalize that much and also shifted more downwards at higher energy efficiency acknowledging the fact that achieving good acc on high energy efficiency is not that easy for the models. so all these show that how flexible and considerate *OTER* is capturing the latest trend with highest intelligence. now let's dive deep into the models that their scores got changed so qwen family models, *i.e.*, M_5 , M_7 , M_{12} showing as highly efficient models got their ratings improved by 1, 2, and 1 respectively. this is fair because already they were showing decent accuracy despite being highly energy efficient, so these are the only models showing this much accuracy at this level of efficiency, so they got upgraded, and the reason M_7 got two ratings upgrade is that so 1 is acceptable and because it's lying on the boundary between two regions and our rating mechsanim is discrete it got improved by two, but still it's fair. so now let's dive into *OTER* for codexglue, let's interpret the change in the curve, so the change from the previous figure is that 5, now it's more of like linear but still the area of ratings classes almost remained similar. so what happened here is that after the new models were added just like previous case, they resonated that the trend of the models

are mostly within the center of accuracy and spanning energy efficiency from lowest to highest, and just by looking at the curve and removing outliers we can think of, mapping an almost linear line can best capture the trend and we see that it did a great job capturing the trend. so the only difference is that in the first when we had less models at lowest energy efficiency, the model expected models to show higher accuracy but now with this intensifying in the middle and spanning whole efficiency spectrum the models convinced the curve that at lowest energy eff is hard to achieve high accuracy so pulled the curve down a bit, so in total the model is more lenient on most of the models, and just like previous RQ that we explained, as improving eff is not as important as acc, because the trend and majority showing so, we see the regions are more indifferent if we go horizontally (eff) and change more rapidly if we go vertically (acc), confirming better acc is harder and should be awarded. and if we get deeper into the models that their rating got shifted, we see 5 models of M_4 , M_9 , M_{10} , M_{17} , and M_{18} all got their rating shifted one class higher, and if we look closely again these models were previously very close to the higher class region but due to the matter of discretization they were lower class and now that we care about accuracy slightly more, those models rating got shifted upwards by one class, but the majority of the trend of the old model seeing is that they are in the original class they were located. also it is fair to the new models added because if it was original curve, the expectation of acc was higher and they might end with lower class at the end, so those consideration are also taken into account so that models with avg accuracy don't get punished. on the other hand, **CIRC** acting as a static and firm had no changes in the initial list of the model ratings, it happened because for the LCB min-max was kept the same and **CIRC** only cares about min-max which by having that it can draw the five circular regions, and for CXG, even though the min-max was changed as earlier explained, since the shift was tiny all models retained their ratings, and new models with their accuracy and efficiency were located properly in between them, so if the scores for the new models are fair to them? yes because in the new set, they still are not close to most efficient and most performant model and we see old models relatively with similar values are getting the same rating with them so it is fair them. so we can say that models are evaluated by their distance to the optimal point, so they are being treated the same as we treated the old models and the distance is sth static computed based on the position of current model compared to optimal one and not sth that change over time to favor one model more than other and also it's important to say that no matter how many models we are dropping in this approach, as long as their values lie between min-max, so we can conclude the **CIRC** rating doesn't change with the number of models in the set. so you see circular regions that are colored 6, any models can be added there and their ratings will be based on the color of circle that they are located at, and the existing ones remain intact and it's fair because all are evaluated with euclidean distance which is fair method in MCDM. DO THEIR SCORES FIT INTO THE BENCHMARK FAIRLY? TODO HERE WHAT DOES CHANGE OF THE RATINGS FOR THE OLD MODELS MEAN ◀

4.3 RQ3: Priority Elasticity

let's dive into 7 first oter-livecodebench, so the pattern that we expect to see is that at (at extreme weights for example we have equal intervals from 0 to 0.2 ... 1). note that the curve of **OTER** won't change, curve is fitted on the same data with the same algorithm. also the ratings boundaries remains monotonic as they were, also we see a gradual increase not drastic ones. at higher weights for energy efficiency the energy efficient models are rated higher. but we can at the same time see that for example even at weight 0.8 not necessarily models with higher energy efficiency wins because models with lower energy eff and higher acc we see they outperform them. also the opposite thing happen at weight energy 0.2 where for example M13 with higher acc still gets lower score than M12, also for example a point that has higher energy eff might start at lower score than a point with . so we are seeing that gradually impact of y or x is vanished and that's why at the end we only care about the absolute value of one point, for example even point being perfectly well on acc while maintaitaning a medium or even high on ene eff, it won't outscore a point with a very bad acc, but slightly higher ene eff, so that's what user is asking us. also the continuoutiy has been preserved. also a point with higher acc and higher eff is always guaranteed to have higher score and we see that. it's also guaranteed

ID	Model	LCB		CXG		LCB Rate		CXG Rate	
		A	E	A	E	CIRC	OTER	CIRC	OTER
M ₁	deepseek-coder-6.7b-base	0.23	0.31	0.49	0.22	2	1	2	3
M ₂	starcoderbase-1b	0	0.93	0.05	0.83	2	1	2	1
M ₃	starcoder2-3b	0.02	0.23	0.4	0.3	1	1	2	3
M ₄	CodeLlama-7b-Instruct-hf	0.21	0.82	0.77	0.22	3	1	3	5
M ₅	Qwen2.5-Coder-3B-Instruct	0.52	0.95	0.39	0.54	4	4	3	3
M ₆	Qwen2.5-Coder-7B-Instruct	0.38	0.78	0.39	0.72	3	2	3	3
M ₇	Qwen2.5-Coder-1.5B	0.48	0.99	0.35	0.84	4	5	3	3
M ₈	deepseek-coder-1.3b-base	0.1	0.76	0.71	0.74	2	1	4	5
M ₉	deepseek-coder-7b-instruct-v1.5	0.58	0.77	0.38	0.33	4	3	2	3
M ₁₀	starcoder2-7b	0.04	0.15	0.58	0.19	1	1	2	4
M ₁₁	codegen-350M-mono	0.02	0.83	0.14	0.78	2	1	2	1
M ₁₂	Qwen2.5-Coder-0.5B	0.27	0.99	0.4	0.84	3	3	3	3
M ₁₃	Yi-Coder-9B	0.33	0	0.21	0	1	1	1	1
M ₁₄	rombos_Replete-Coder-Llama3-8B	0.21	0.74	0.24	0.27	3	1	2	2
M ₁₅	speechless-code-mistral-7b-v1.0	0.31	0.85	0.4	0.57	3	2	3	3
M ₁₆	stable-code-3b	0.06	0.68	0.55	0.64	2	1	3	4
M ₁₇	CodeLlama-7b-Python-hf	0.15	0.73	0.41	0.22	2	1	2	3
M ₁₈	Seed-Coder-8B-Instruct	1	0.88	0.34	0.56	5	5	3	3
M ₁₉	CodeQwen1.5-7B-Chat	0.06	0.99	0.08	0.95	2	1	2	1
M ₂₀	Magicoder-S-DS-6.7B	0.42	0.62	0.42	0.4	3	2	3	3
M ₂₁	granite-8b-code-base-4k	0.1	1	1	0.13	2	1	2	5
M ₂₂	codegen-2B-mono	0.02	0.64	0.48	0.48	2	1	3	3
M ₂₃	speechless-zephyr-code-functionary-7b	0.21	0.79	0.45	0.19	3	1	2	3
M ₂₄	OpenCoder-8B-Instruct	0.31	0.1	0.43	0.15	1	1	2	3
M ₂₅	llama-3.2-1b-code-instruct	0.08	0.98	0.19	0.8	2	1	3	2
M ₂₆	OpenCodeInterpreter-DS-6.7B	0.42	0.64	0.42	0.24	3	2	2	3
M ₂₇	mini-coder-1.7b	0.31	0.61	0.22	0.62	3	1	2	2
M ₂₈	codegemma-2b	0.1	0.96	0.26	0.81	2	1	3	2
M ₂₉	Qwen3-1.7B-Sushi-Coder	0.54	0.61	0.24	0.62	3	2	2	2
M ₃₀	CodeNinja-1.0-OpenChat-7B	0.15	0.21	0.39	0.44	1	1	3	3
M ₃₁	qwen2.5-coder-3b-claude_opus_4.6	0.04	0.53	0.41	0.53	2	1	3	3
M ₃₂	codegemma-7b-it	0.33	0.82	0.46	0.46	3	2	3	3
M ₃₃	prism-coder-7b	0.08	0.92	0	1	2	1	2	1

Table 2. Summary of model performance on LiveCodeBench (LCB) and CodeXGLUE (CXG): normalized accuracy (*A*), normalized efficiency (*E*) and unified ratings (CIRC and OTER) for each benchmark.

that if two models are showing (x_1, y_1) and (x_2, y_2) where $x_2 \geq x_1$, and $y_2 \leq y_1$, and the rating of second model is better than first model, if we increase energy weight the second model will try to catch the first model and for sure will catch it at the end and the same process happens for when we are increasing weight of accuracy. as we can see all our requirements have been met.

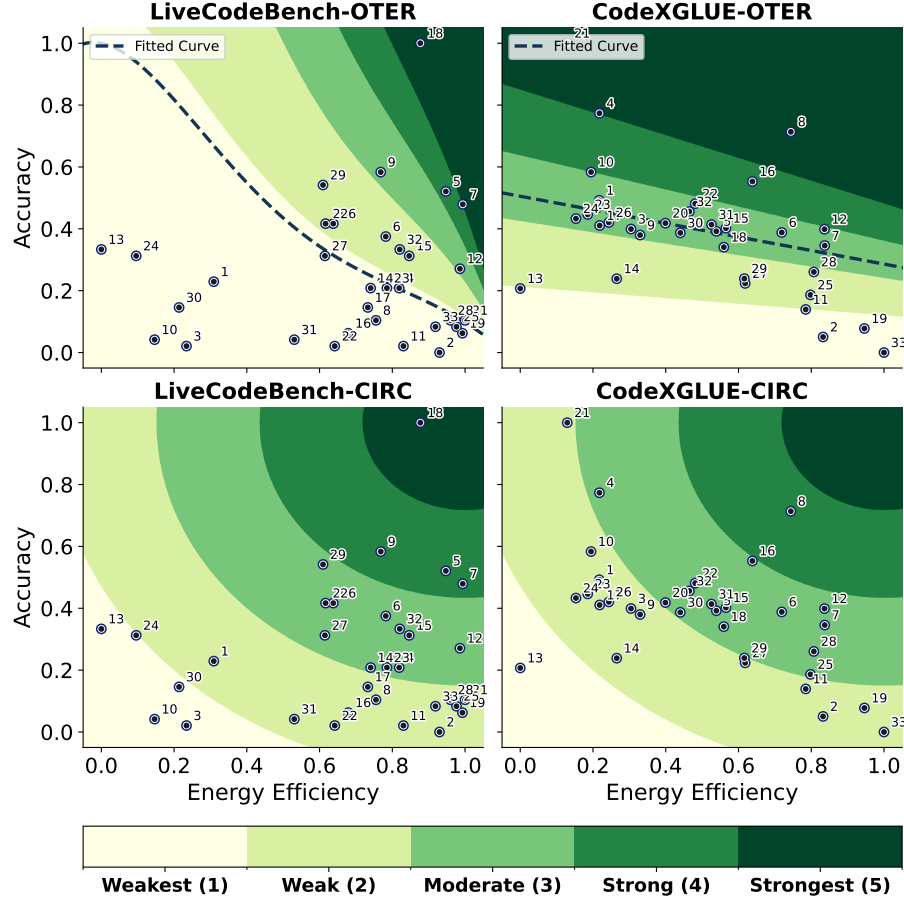


Fig. 6. Unified energy-accuracy model rating; numbers in the plot show models' IDs

now lets dive into the figures: first row when weight energy is 0, so all we care about is the accuracy, so that's why you see a horizontal line only separating models by their accuracy, so in a region as you move forward increasing energy efficiency you wont see any difference in rating, that's what it means to have all priority on acc. you can see as we move forward towards higher energy efficiency weights those boundaries start to care more about energy efficiency, in a way that they try to score the points low energy efficiency less although they were previously at $w_e = 0$ had higher ranking, they get rank 1, and for the points at extreme cases with high energy efficiency with low acc that previously all were ranked 1, now their rank increase gradually. the shape of boundaries depends on the curve fitted such that when weights are near 0.5 the final boundary curve will look like the baseline curve and boundaries with weight $e = 0.5$, and also we see that as we approach towards min or max weight the area of boundary within each region must be approximately equal to other regions while in center of the weight, it will depend mostly on the complexity of $y \times \frac{1}{f(x)}$. the same thing said for the first row also holds the third row which shows **OTER** on CXG. but the thing to notice is that before w energy is less than 0.5, we see more of like linear separation of boundaries, this happens since our formula turns into $\frac{y}{f(x)^{2w_e}}$ and as $f(x)$ is linear the exponent of that (less than 1) will almost look linear. however as

w_e gets more than 0.5, our formula turns into $x \times f(x)^{-2w_a} \times y^{2w_a}$, the this terms $x \times f(x)^{-2w_a}$ will cause the model to form mostly like a new curve that is not linear so that's why see bent curve. but the overall thing we see is that increasing w_e causes the boundaries care about more energy efficiency that's why see that the linear initial horizontal lines are turning into linear that score models with low energy efficiency lowers and at higher energy efficeicny the boundaries drop because models are showing good ene eff so their score should be better, and at the end we see that for example two models that got highest score in the baseline weight now they got score of 4 and 2 because showing not good ene eff. and again in the model you see caring more about acc and ene at the same time while at extreme weights you see more inclinations towards one aspect.

the CIRC ones you see regardless of the position of the models, we got the same pattern of how boundaries are changing. and how they are changing is actually driven by the equation defined in 11 so for accuracy prioritization our goal finally is to have a horizontal lines separating boundary and that' why you see that when acc weight is higher you got ellipse that is stretched towards the horzional (x) axis caring more about acc. how? so as you stretch the accuracy, you'll see that during high spectrum of energy efficeicny your rating remains the same which is ideally what we want and also you see that with small increment or increase in accuracy your rate will change fast. on the other hand, when you have all attention to energy, so at the end we want a flat vertical boundaries separating them, so this thing you see happened at extreme energy, where whatever you change acc the rating won't change, and changes only happen with energy change. also you see that when weight energy is more than 0.5, we got a boundaries more close to vertical boundaries and here unlike previous case small improvement in energy will actually lead to higher score, but very large improvement in acc can only change the score, and as you see for example for $w_e = 0.2$ when the points literally had energy efficeicny of 0.2 could get score of 4 at max, and now with $w_e = 0.8$ points with energy efficiency of 0.2 can only get score of 2 at max, so that's what we want caring more about energy. and for OTER plot we can see the same pattern for example when energy efficiency weight is 0.2 they can gete score of 5 but when that weight increases their score gradually decrease and for energy efficeicny weight of 0.2 their score could maximum reach 3 at acc of highest.

interpreting 8, so here what we are doing is that we compare the adjacent correlation so that to make sure there is not a drastic change when we are changing the weights, this step is just for validation of our approach, so we also plotted the correlation of that specific point with the baseline where weights are equal to show that adjacent is mostly higher than correlation baseline with current one.

the reason that you see that when energy weight is at its maximum and correlation of baseline is dropping more for **OTER** is because as we already explained in 4.2 achiving energy efficiency is not considered that much hard for the models so when we have w_e at its maximum the energy will only be considered while baseline saying that energy achievement is not hard, so that's why we see a low correlation in LCB and even much lower at codexglue which proved to be much more caring about the acc. also we should observe a trend that as we start from 0 and reach 0.5 the baseline corerelation with the current weight should gradually get higher and as we get further from 0.5 and reaching 1, again the correlation with baseline should deminish, which is observable in all of the 4 figures in 8. so we always see adjacent ratings are not chaniging drastically and the correlation remains above 0.8 for all 4 plots. and as obvious for **CIRC** we see more symmetry as it equation is a static formula changing the regions and ratings predictably as we change the weights.

interpreting 9 so in total we get the exact behavior we expect, energy dominant score should drop when we increase the priority of accuracy on the other hand, acc dominant should prevail and get better and better, and this pattern can be obviously seen in all plots.

for all methods we see for balanced models their score relatively remains the same which is just as expected. for livecodebench we see almost smooth transition for both energy dominant and acc dominant because the slope of the curve fitted is not flat and here energy efficiency achievement has a decent compromise on accuracy, so a model with energy efficiency of 0.9 with equal weights distribution (0.5 and 0.5) can be seen that even though it has lower score than the energy dominant one but its score is closer to that compared to codexglue one. also the curve that we see in both codexglue and livecodebench depends the curve fitted if the curve fitted is almost linear so we see more linearity in these curve here. but the general thing that we should see is that acc dominant starts at lowest score when all priority is on energy weight but its score gradually improves when priority of acc gets higher until it reaches the maximum point and the opposite thing exactly happens for energy dominant one. for codexglue for span from 0 to 0.5 : the reason energy dominant one drops so fast is that because at equal weights, model saying because we already discussed that in figure 2 that energy eff achievement is not high so it's score is low, so the drastic slope you see from $w_a = 0$ to 0.5 for score of this energy-dom is that model should actually make the model from the best one when $w_a = 0$ to one of the worst one when $w_a = 0.5$ so that's why we see a instant slope, and the rest is more of like flat because model was already at its among lowest scores so it can't decrease further. (discussed more in next paragraph) and from 0.5 to 1, since now our peicewise function swaps and care about accuracy, so for example you see acc dominant increases linearly, because as you in the 7 the curve is linear as well and since we have $\frac{y}{f(x)^{2w_e}}$ for $w_a > 0.5$, so actually in 6 we discussed that even when there was equal weights for codexglue that increase in energy efficiency didn't cause that much score change from the models, and the scores for energy efficient models was already low so now we see increasing $w_a > 0.5$, can't decrease the score of energy efficient model as it's already low near zero, but for acc dominant model the whole score will be controlled by y and $f(x)$ to the power is approaching zero, so that's why we see almost linear improvement and that's why for example for codexglue we see that when $w_a = 0.5$, the acc dominant one already covered a good part of the scroing and reached till 0.8 so it showing the curve in baseline mode choose accuracy as hard to achieve, in the lcb we also see the same pattern for example for the acc dominant it's outperforming energy dominant in $w_a = 0.5$, but since in that plot energy efficiency was still important they are closer and we see a more slope decline. so overall the curve and the thing we are seeing is caused by weighted function of **OTER** defined in 18.

and for **CIRC** since it's a static approach with fixed points drawing concentric circles we will have the same plot for both benchmarks and we can observe that balanced point almost remain in the center of scoring while those dominant ones changes their score with respect to formula defined in the equation of 11 consistently reducing or imporving their score as the weight of accuracy is increasing.

4.4 RQ4: Scaling Laws Efficiencies

To answer RQ4, we examine how efficiently each CLM converts its physical footprint into task accuracy (*Performance Efficiency*, ψ) and into energy savings (*Structural Efficiency*, τ), using the size–accuracy and size–energy scaling curves introduced in Section 3.6. Table 3 reports the size, raw accuracy, raw energy, and the resulting ψ and τ ratings for all models on both benchmarks, while Figures 10 and 11 visualize the corresponding rating regions.

Why size belongs in the evaluation. Figures 10 and 11 make clear why incorporating size is essential: models of nearly identical footprint exhibit markedly different accuracy and energy. In the size–accuracy view, the fitted curve encodes the accuracy expected at each size. Models lying *above* it extract more capability than their scale predicts—evidence of a stronger architecture, higher-quality or more plentiful training tokens, or better post-training—whereas models *below* it under-utilize their parameters, pointing to token starvation, architectural inefficiency, or saturation that additional parameters no longer relieve (such models are candidates for pruning or for genuinely novel architectural

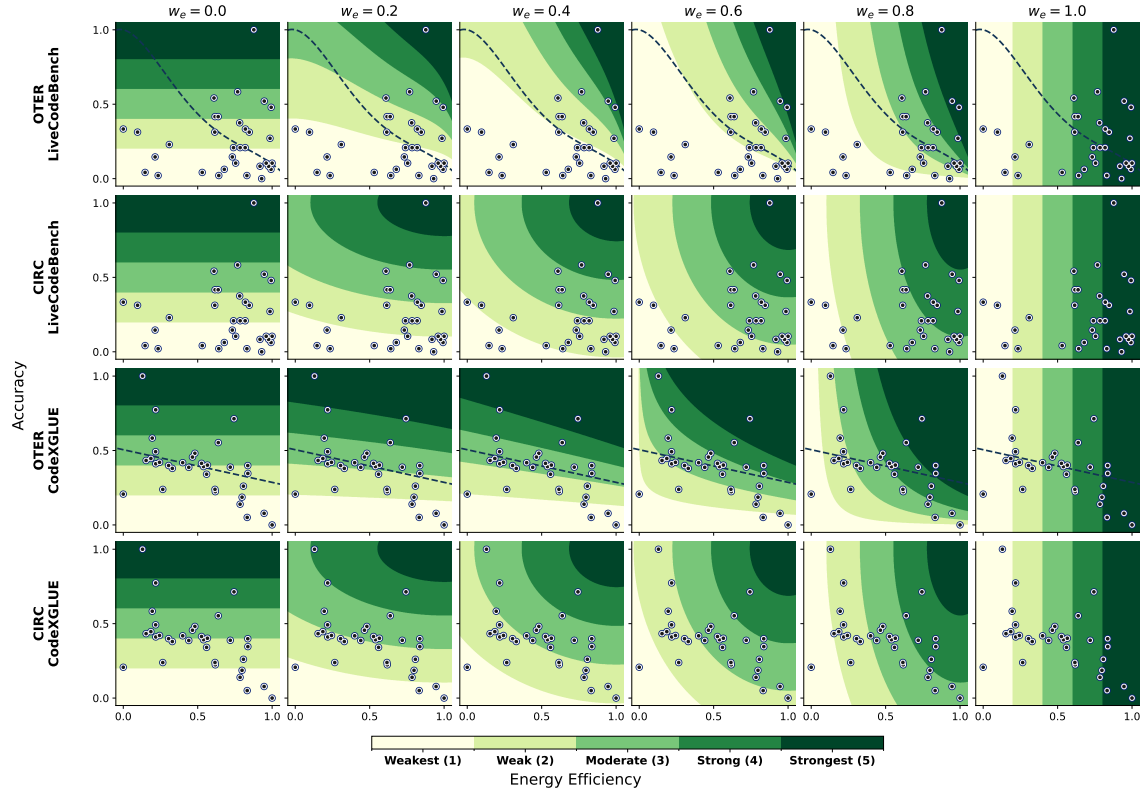


Fig. 7. Impact of weight increase on models scores

changes rather than further scaling). The size–energy view is even more dispersed: at a fixed size, total energy varies by more than an order of magnitude. Around 13–16 GB, for example, LiveCodeBench energy ranges from 7.5×10^4 J for granite-8b-code-base-4k to 1.78×10^6 J for Yi-Coder-9B. Models below the energy curve are structurally efficient—either architecturally lean or disciplined about when to stop—while models above it over-consume, whether through heavier internal computation or runaway generation when they cannot converge on an answer. The two curves therefore turn “size” from a confound into a diagnostic baseline.

Model size alone does not dictate efficiency. Our first observation is that a small footprint is not inherently rewarded, nor is a large one inherently penalized; what matters is how efficiently a model uses the parameters it has. Statistically, size and rating are essentially uncorrelated (Spearman $\rho = 0.15$, $p = 0.42$ for Performance Efficiency, and $\rho = 0.04$, $p = 0.83$ for Structural Efficiency), and the average model size within each rating bucket exhibits no monotonic ordering. On LiveCodeBench, for instance, the rating-5 Performance-Efficiency group averages only 5.59 GB while the rating-2 group averages 14.20 GB, yet the rating-3 group averages 7.88 GB; on CodeXGLUE the average size is nearly constant across all five Performance-Efficiency ratings (8.75–10.55 GB), confirming that the score is decided by architectural density rather than scale. Two extremes make this concrete. The smallest model in our cohort, codegen-350M-mono (M_{11} , 0.74 GB), ranks among the weakest along every axis on both benchmarks—size–accuracy, size–energy, and the energy–accuracy ratings of RQ1—showing that a minimal footprint buys nothing once accuracy collapses. Conversely,

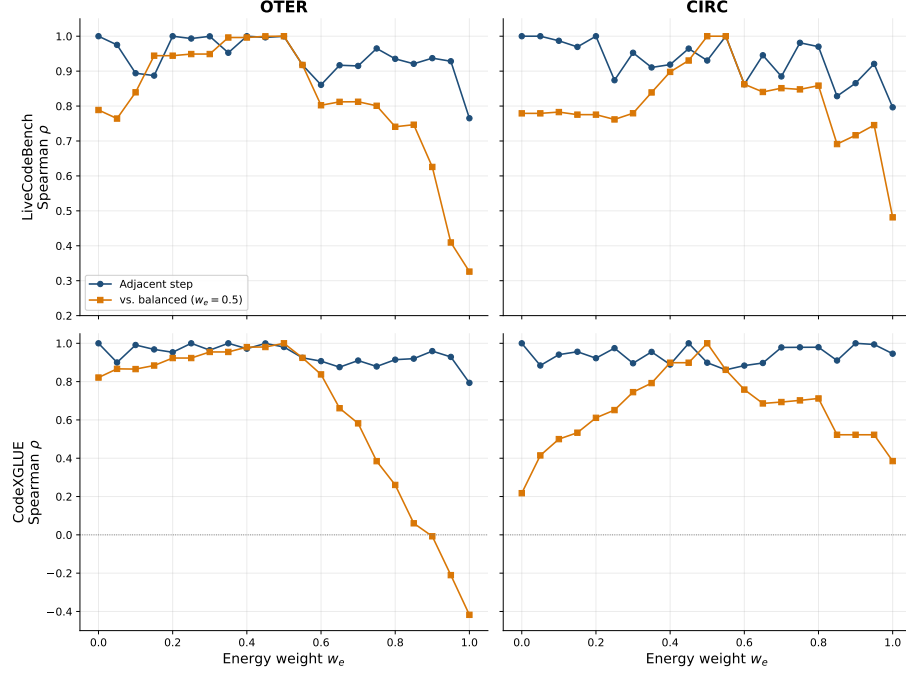


Fig. 8. Comparing the continuity of rankings between adjacent weights and baseline weights.

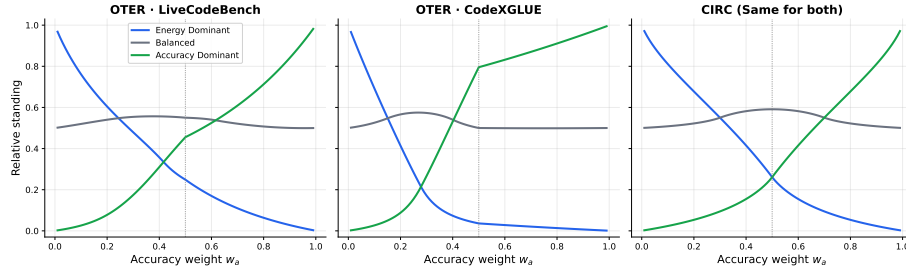


Fig. 9. Scores of hypothetical dominant-objective (eff = 0.9 and acc = 0.1 or vice versa) and average-performance models (eff = 0.5 and acc = 0.5) with gradual weight increase.

Seed-Coder-8B-Instruct (M_{18} , 15.37 GB) attains the top rating on both size-adjusted axes on LiveCodeBench ($\psi = 5$, $\tau = 5$), demonstrating that a comparatively large model can still translate its parameters into accuracy while staying below the energy its size would predict. Consistent with this, a majority ($\approx 62\%$) of the below-median-size models (≤ 12.6 GB) fall below the size-accuracy curve—under-performing the accuracy expected at their scale—and a similar majority sit above the size-energy curve, consuming more energy than their size warrants. The latter can be a symptom of small models that fail to halt and keep generating tokens until the limit is reached.

The geometry of the curves also encodes a deliberate, complementary trade-off. Because the size-accuracy curve is increasing, two models with identical accuracy are scored differently: the smaller one deviates further above the curve

ID	Model	Size (GB)	LCB		CXG		LCB Rate		CXG Rate	
			A	E	A	E	ψ	τ	ψ	τ
M ₁	deepseek-coder-6.7b-base	12.56	0.07	1.25e6	0.09	1.23e6	2	2	3	1
M ₂	starcoderbase-1b	4.24	0	1.95e5	0.01	2.68e5	1	5	1	4
M ₃	starcoder2-3b	11.29	0.01	1.38e6	0.08	1.09e6	1	1	3	2
M ₄	CodeLlama-7b-Instruct-hf	12.55	0.07	3.84e5	0.15	1.23e6	2	4	4	1
M ₅	Qwen2.5-Coder-3B-Instruct	5.75	0.17	1.65e5	0.08	7.25e5	4	5	3	2
M ₆	Qwen2.5-Coder-7B-Instruct	14.19	0.12	4.47e5	0.07	4.45e5	2	4	2	4
M ₇	Qwen2.5-Coder-1.5B	2.88	0.15	8.57e4	0.07	2.61e5	5	5	3	4
M ₈	deepseek-coder-1.3b-base	2.51	0.03	4.93e5	0.14	4.06e5	2	3	5	3
M ₉	deepseek-coder-7b-instruct-v1.5	12.87	0.19	4.70e5	0.07	1.05e6	4	4	2	2
M ₁₀	starcoder2-7b	13.36	0.01	1.53e6	0.11	1.26e6	1	1	3	1
M ₁₁	codegen-350M-mono	0.74	0.01	3.64e5	0.03	3.43e5	1	2	2	2
M ₁₂	Qwen2.5-Coder-0.5B	0.92	0.09	9.90e4	0.08	2.62e5	5	5	3	3
M ₁₃	Yi-Coder-9B	16.45	0.11	1.78e6	0.04	1.57e6	2	1	2	1
M ₁₄	rombos_Replete-Coder-Llama3-8B	14.96	0.07	5.17e5	0.05	1.15e6	2	4	2	2
M ₁₅	speechless-code-mistral-7b-v1.0	13.49	0.1	3.37e5	0.08	6.84e5	2	5	3	3
M ₁₆	stable-code-3b	5.21	0.02	6.23e5	0.11	5.71e5	1	3	4	3
M ₁₇	CodeLlama-7b-Python-hf	12.55	0.05	5.31e5	0.08	1.23e6	1	4	3	1
M ₁₈	Seed-Coder-8B-Instruct	15.37	0.32	2.85e5	0.07	6.93e5	5	5	2	3
M ₁₉	CodeQwen1.5-7B-Chat	13.5	0.02	8.73e4	0.02	9.13e4	1	5	1	5
M ₂₀	Magicoder-S-DS-6.7B	25.11	0.13	7.29e5	0.08	9.44e5	2	4	2	3
M ₂₁	granite-8b-code-base-4k	15	0.03	7.49e4	0.19	1.36e6	1	5	5	1
M ₂₂	codegen-2B-mono	5.3	0.01	6.86e5	0.09	8.16e5	1	3	3	2
M ₂₃	speechless-zephyr-code-7b	13.98	0.07	4.41e5	0.09	1.28e6	2	4	3	1
M ₂₄	OpenCoder-8B-Instruct	14.48	0.1	1.62e6	0.08	1.33e6	2	1	3	1
M ₂₅	llama-3.2-1b-code-instruct	2.3	0.03	1.15e5	0.04	3.22e5	1	5	2	4
M ₂₆	OpenCodeInterpreter-DS-6.7B	12.56	0.13	6.93e5	0.08	1.19e6	3	3	3	1
M ₂₇	mini-coder-1.7b	3.2	0.1	7.32e5	0.04	6.02e5	3	2	2	2
M ₂₈	codegemma-2b	4.67	0.03	1.43e5	0.05	3.08e5	1	5	2	4
M ₂₉	Qwen3-1.7B-Sushi-Coder	3.2	0.17	7.41e5	0.05	6.05e5	5	2	2	2
M ₃₀	CodeNinja-1.0-OpenChat-7B	13.49	0.05	1.42e6	0.07	8.80e5	1	1	2	3
M ₃₁	qwen2.5-coder-3b-claude_opus_4.6	5.75	0.01	8.75e5	0.08	7.46e5	1	2	3	2
M ₃₂	codegemma-7b-it	15.9	0.11	3.81e5	0.09	8.41e5	2	5	3	3
M ₃₃	prism-coder-7b	13.04	0.03	2.14e5	0	7.38e3	1	5	1	5

Table 3. Summary of models' efficiency scores on LiveCodeBench (LCB) and CodeXGLUE (CXG): accuracy (A), energy consumption (E) and unified ratings, i.e., $\psi(\text{accuracy}, \text{size})$ and $\tau(\text{energy}, \text{size})$, for each benchmark.

and earns the higher Performance Efficiency rating. Symmetrically, because the size–energy curve is increasing, two models with identical energy are separated in favor of the *larger* one, which sits further below its (higher) expected energy and earns the higher Structural Efficiency rating. Formally, for two models $m_1 = (A, E, S_1)$ and $m_2 = (A, E, S_2)$ with $S_1 < S_2$, m_1 wins on size–accuracy while m_2 wins on size–energy. In words, Performance Efficiency rewards reaching a given accuracy with fewer parameters, whereas Structural Efficiency penalizes a small model for spending as much energy as a larger one. We further observe that the size–accuracy curve is nearly flat across the cohort: small models already reach the highest observed accuracies, so within this regime, added size yields only marginal accuracy

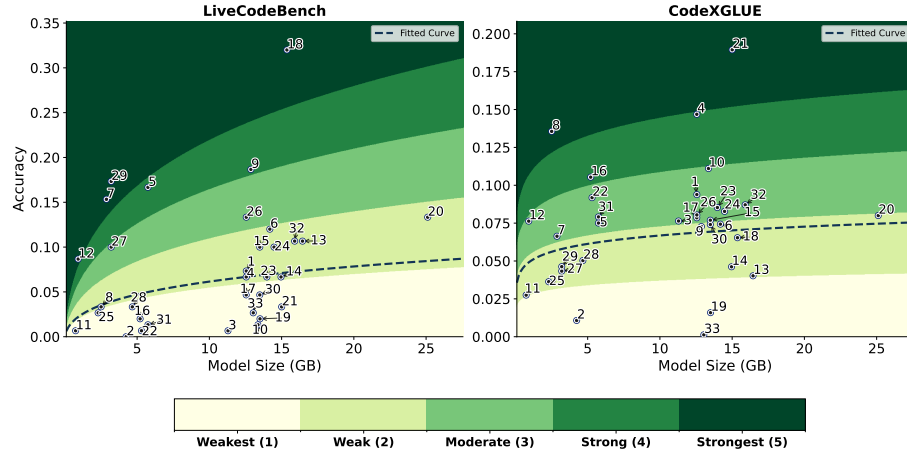


Fig. 10. Unified size-accuracy model rating; numbers in the plot show models' IDs.

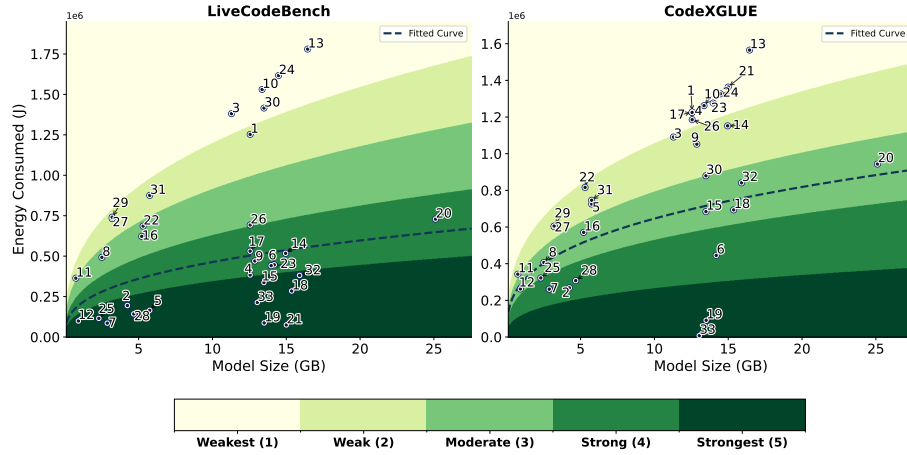


Fig. 11. Unified size-energy model rating; numbers in the plot show models' IDs.

gains—the curve permits a large increase in size for a slight accuracy improvement. Finally, deviation from the curves grows with scale, most visibly for energy: the largest models show the widest spread, consistent with a greater tendency to over-generate or to have been trained for longer contexts.

Joint behavior across the two efficiency axes. Examining ψ and τ together reveals that, on both benchmarks, a model is more likely to excel on one axis while lagging on the other than to succeed or fail on both. On LiveCodeBench, only 5 models are strong on both ($\psi, \tau \geq 4$) and 8 are weak on both ($\psi, \tau \leq 2$), while 15 split between a high and a low rating. On CodeXGLUE, *no* model is simultaneously performance- and structurally efficient, 6 are weak on both, and 8 split. The complete absence of dual-efficient models on CodeXGLUE underscores how difficult it is, in code summarization, to produce an adequate docstring while keeping energy low for the model's size; pairing accuracy gains with structural restraint appears especially hard for this task. The models weak on both axes (8 on LiveCodeBench, 6 on CodeXGLUE)

under-deliver accuracy for their size while over-spending energy. For the larger members of this group this may indicate structural bloat—added parameters that neither improve accuracy nor better energy efficiency—whereas for the smaller members (M_{11} and M_{31} among them) it may reflect architectural or algorithmic inefficiency and a tendency to over-generate rather than any capability gained from scale.

The starkest one-good-one-bad behavior appears in three models—CodeQwen1.5-7B-Chat (M_{19}), granite-8b-code-base-4k (M_{21}), and prism-coder-7b (M_{33})—each showing the maximum possible rating gap (four classes) between the two axes on *both* benchmarks. On LiveCodeBench all three are similar in size (≈ 13 – 15 GB) and consume far less energy than their size predicts (earning $\tau = 5$), yet their accuracy is so low that they are heavily penalized on size-accuracy ($\psi = 1$); numerous smaller models match or exceed their accuracy. This profile suggests architectures that are internally lean—or that simply do not over-generate—but fail to solve the task, possibly because they do not understand it well enough to produce useful tokens.

Individual models illustrate the full range of trade-offs the two axes expose. starcoderbase-1b (M_2) is efficient *only* structurally: with near-zero accuracy it expends very little energy—less than other models of its size—so it scores best on size-energy ($\tau = 5$ on LiveCodeBench) but worst on size-accuracy ($\psi = 1$), having effectively traded capability for a small, low-energy profile. Qwen3-1.7B-Sushi-Coder (M_{29}) is the mirror image: it achieves strong accuracy for a 3.2 GB model ($\psi = 5$) but draws far more energy than its size warrants, sitting above the size-energy curve ($\tau = 2$), so the accuracy it buys is not worth its structural cost. Qwen2.5-Coder-0.5B (M_{12} , 0.92 GB) shows why size context changes the verdict: judged on energy and accuracy alone it is merely moderate (rating 3 under both **CIRC** and **OTER** on LiveCodeBench) because its accuracy is low, yet *relative to models of its size* it is both more accurate and more energy-frugal, earning $\psi = 5$ and $\tau = 5$. granite-8b-code-base-4k (M_{21}) on CodeXGLUE is the opposite extreme: it is exceptionally accurate (top energy-accuracy rating of 5 under **OTER**, 2 under **CIRC**) and clearly outperforms similar-size models on size-accuracy ($\psi = 5$), but it consumes far more energy than its size predicts, collapsing to $\tau = 1$. Even the largest model is not automatically penalized: Magicoder-S-DS-6.7B (M_{20} , 25.11 GB) is a clear size outlier yet earns a good-to-moderate Structural Efficiency rating ($\tau = 4$ on LiveCodeBench, 3 on CodeXGLUE) because it does not consume more energy than much smaller models, signaling a disciplined and efficient architecture, while remaining weak on size-accuracy ($\psi = 2$) since smaller models reach comparable accuracy. Finally, deepseek-coder-1.3b-base (M_8) on CodeXGLUE sits almost exactly on its expected energy ($\tau = 3$) while exceeding the accuracy expected for its size ($\psi = 5$), a clean example of a small footprint converted into above-trend performance.

Robustness and generalization. Several analyses confirm that the size-adjusted ratings are stable and that the underlying curves generalize. Leave-one-out analysis shows that removing any single model barely perturbs the rankings: Performance Efficiency drifts by 0.0398 on average (Kendall- $\tau = 0.9882$) and Structural Efficiency by 0.0256 (Kendall- $\tau = 0.9909$). Under 5% uniform input perturbation, mean absolute rank drift stays below 0.41 for Performance Efficiency and 0.04 for Structural Efficiency. To probe whether the curves can extrapolate to unseen, larger models, we refit each curve on the smaller (denser) half of the cohort and predict the held-out larger (sparser) half. The error grows only modestly—accuracy RMSE rises from 0.068 in-sample to 0.086 extrapolated, and energy RMSE from 4.80×10^5 to 5.69×10^5 J, an 18% increase that is acceptable given the magnitude of the values—indicating that the curves remain predictive for footprints larger than those used to fit them, generalizing beyond the fitted size range rather than overfitting it. Overall, the size-adjusted metrics are robust to measurement noise, and they genuinely reward architectural density over brute-force capacity.

Summary

- Model size is orthogonal to efficiency: the smallest model is among the weakest and a comparatively large model is the strongest, with no monotonic size–rating relationship (Spearman $\rho \leq 0.15$, $p > 0.4$).
- Performance Efficiency rewards reaching a given accuracy with fewer parameters, while Structural Efficiency rewards staying below the energy a model’s size predicts—so for equal accuracy/energy the smaller model wins on size–accuracy and the larger on size–energy.
- Models overwhelmingly excel on one efficiency axis while lagging the other; no model is dual-efficient on CodeXGLUE, highlighting the difficulty of pairing accurate generation with structural restraint.
- The fitted curves are stable (leave-one-out and noise tests) and extrapolate to larger models with only modest error, supporting their use as fair, size-aware baselines.

5 Discussion

Characteristics of CIRC vs. OTER: Based on the characteristics of *CIRC* and *OTER*, both approaches can effectively solve the MCDM problem, though their practicality depends on the users’ criteria. *CIRC* allows objective compensation without heavily penalizing low performance in one, favoring both equally—ideal when objective correlation is unknown. It provides robust scoring by fixing an optimal point and ranking by distance. Pre-defined class ranges ensure consistent scoring regardless of other samples.

OTER on the other hand, is a trend-aware mechanism that rewards models performing better than expected. Hence, depending on the expectation curve, it can reward models excelling in one objective, demonstrating fairness, and penalize those with moderate performance in both objectives, demonstrating robustness. A key advantage of *OTER* is its ability to reveal each model’s true capacity, highlighting outliers and guiding further improvement, as reflected in the unbounded raw scores, *i.e.*, $0 \leq v \leq \infty$.

Inclusion of New Models: *CIRC* method remains consistent when new models are added. Specifically, if the new data’s accuracy or energy efficiency falls within existing ranges, scoring remains unchanged. However, if it exceeds current limits, the scores must be recalculated to ensure fair scoring.

Similar to *CIRC*, *OTER* provides a fair score when new data points are added to our dataset. However, since it is a trend-aware and pre-normalized method, it is recommended to refit the model to ensure consistent and fair comparisons across models.

Inclusion of New Benchmarks: We reported ratings separately for the two software benchmarks. However, adding benchmarks causes rating variations that complicate comparison. Thus, we recommend running BRACE on each and averaging the ratings.

6 Implications

BRACE empowers users of LLMs and AI technologies, in our context software development teams, to take actionable steps toward improving their workflows and adopting AI solutions that are both accurate and energy-efficient. *Model hosting platforms* such as HuggingFace can integrate the proposed energy–accuracy ratings alongside their listed models to promote more informed and sustainable adoption choices. *Software developers* and *GenAI practitioners* can confidently select the most optimal models for their projects, balancing performance with environmental impact.

Similarly, companies can leverage these ratings during product development to position themselves as both performance-driven and environmentally conscious, thereby attracting customers. *Researchers* can also use the framework to analyze architectural differences, identify factors influencing energy-accuracy trade-offs, and uncover gaps across models for future improvement.

Although our implementation focuses on *CLM*, the framework can be readily extended to domain-specific models, enabling fairer comparisons within other fields. Furthermore, it facilitates model benchmarking for *publishers*, helping them understand where their model stands in the broader landscape.

7 Related Works

General LLM Benchmarking: Numerous studies benchmark LLMs based on various criterias [96, 108]. For example, IFEVAL [109] evaluates models on instruction-following abilities using 25 verifiable tasks. Similarly, HELM [59] provides a holistic bench measuring accuracy, calibration, robustness, fairness, bias, toxicity, and efficiency for 30 models.

Sustainability Behavior Analysis: In parallel to evaluating LLMs on different benchmarks, studying the sustainability behavior of current LLMs and optimizing them have been widely investigated in recent literature [62, 71, 90]. Morrison *et al.* [73] holistically measured the resource exhausted throughout the development phase. Moreover, Google and Meta disclose their carbon footprint during their pretraining stages for the Gemma and Llama models, respectively [98, 100]. Other studies [82, 89] have examined the impact of optimization techniques on the energy consumption of AI pipeline stages, ranging from pre-training to inference phases.

LLM Sustainability and Benchmarking:

Several studies have focused on benchmarking LLMs on their environment impact. Argerich *et al.* [11] developed an energy profiler to measure energy consumption of an LLM during inference. Rajput *et al.* [83] benchmarked quantization techniques from an energy-efficiency perspective, providing insights for choosing sustainable models in resource-constrained settings. Samsi *et al.* [100] analyzed Llama variants (7B, 13B, 65B) across different hardware and configurations to highlight the benefits of power capping and improved GPU utilization. Ecologits [36] proposed a framework to estimate the environmental impact of proprietary models using regression-based energy and latency predictions. HuggingFace AI energy score benchmarked both proprietary and open-source models on ten tasks and reported 1-5 star ratings for their energy performance [63].

Joint Accuracy and Energy Benchmarking: As already discussed, benchmarking LLMs only on green AI perspective overlooks the functional correctness, making their adoption challenging since their true performance remains unexplored. We observe a few works targeting both accuracy and energy aspects. Luccioni *et al.* [64] evaluated whether a model exhibits comparable performance regarding its emissions.

ZEUS [106] experimented with various language models and tasks and formulated a leaderboard with energy-accuracy related metrics. Other works study Pareto-optimal energy-accuracy trade-offs in specialized domains: Alizadeh *et al.* [7] focus on SE tasks, while Husom *et al.* [47] target embedded systems. Lastly, in the domain of scoring and ranking LLMs from a sustainability perspective, Pham *et al.* [80] scrutinize sustainability, correctness, and computation characteristics of Small Language Models (SLM). Kaplan *et al.* [54] proposed a novel metric, *i.e.*, delta carbon emission by computing the ratio of normalized carbon emission and f1-score. Jegham *et al.* [50] proposed a framework to predict sustainability data of proprietary models and used cross-efficiency DEA to rank models by performance relative to environmental cost.

Despite these advancements in the green AI field, a sound approach that jointly considers both energy and accuracy while capturing their correlated behavior remains lacking. Moreover, the sustainability aspects of *CLMs* remain underexplored, limiting our ability to systematically compare their energy-accuracy trade-offs. Our framework addresses these gaps by providing seamless energy and accuracy computation on two widely used software engineering benchmarks. We introduced two tailored rating methods that classify models on a 1–5 scale based on a fair and robust assessment of both energy and accuracy that can be applied to diverse use cases. We then benchmarked all *CLMs* using these approaches to reveal the strengths and weaknesses of each model.

8 Threats to Validity

Internal Validity:

Several design factors could introduce measurement variations. For energy measurement reliability, we followed best practices [85], complied with warm-up and cool-down procedures, executed each experiment three times, configured CodeCarbon to *process* mode for isolation, and cross-validated with *perf* CLI. We measured energy only for LLM operation blocks, excluding overhead. To prevent GPU/CUDA cache residues from affecting measurements, we executed each model in separate processes.

External Validity:

Factors like benchmark subset selection and hardware differences may limit generalizability. However, we selected a reasonably representative subset with documented selection criteria. Model outputs can differ across machines with identical seeds and deterministic CUDA algorithms due to 16-bit precision—inherent precision losses handled differently by machines/CUDA versions. We maintained original precision settings for consistent, fair conditions. Moreover, a model’s energy consumption can vary across machines due to differences in hardware architectures and computational capacity. We conducted measurements on two different machines and observed a coherent and reasonable shift in energy consumption. Because these shifts are consistent and accuracy remains comparable, overall rating trends are expected to be stable across machines.

Construct Validity:

Both approaches normalize energy and accuracy using minimum and maximum values, where extreme values can distort relative positions. However, outlier removal and a large evaluated model set with consistent ratings support reliability.

9 Conclusions

We propose BRACE to benchmark *CLMs* in terms of functional correctness and energy efficiency. We further introduce two systematic rating methods, **CIRC** and **OTER**, that score *CLMs* along unified accuracy-energy dimensions. Our experiments on widely used state-of-the-art code-to-text and text-to-code benchmarks show that **CIRC** provides a stable, deterministic ranking, whereas **OTER** is trend-aware, robust, and more sensitive to outliers. Overall, the resulting ratings act as portable energy–accuracy tags, helping software and AI practitioners identify models that achieve strong correctness at low energy cost.

References

- [1] Mansour Zoubeirou a Mayaki and Victor Charpenay. 2025. Modeling Energy Consumption in Deep Learning Architectures Using Scaling Laws. (2025).
- [2] Mohammad Abdollahi, Ruixin Zhang, Nima Shiri Harzevili, Jiho Shin, Song Wang, and Hadi Hemmati. 2025. Surveying the Benchmarking Landscape of Large Language Models in Code Intelligence. (2025).

- [3] Saima Afrin, Joseph Call, Khai-Nguyen Nguyen, Oscar Chaparro, and Antonio Mastropaolo. 2025. Resource-Efficient & Effective Code Summarization. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE.
- [4] Toufique Ahmed and Premkumar Devanbu. 2022. Few-shot training LLMs for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering*.
- [5] Dennis Aigner, CA Knox Lovell, and Peter Schmidt. 1977. Formulation and estimation of stochastic frontier production function models. *Journal of econometrics* 6, 1 (1977).
- [6] Ibrahim M Alabdulmohsin, Behnam Neyshabur, and Xiaohua Zhai. 2022. Revisiting neural scaling laws in language and vision. *Advances in Neural Information Processing Systems* 35 (2022), 22300–22312.
- [7] Negar Alizadeh, Boris Belchev, Nishant Saurabh, Patricia Kelbert, and Fernando Castor. 2025. Language Models in Software Development Tasks: An Experimental Analysis of Energy and Accuracy. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE.
- [8] Abdullah Mahmoud Almasri and Luis Borges Gouveia. 2021. Star-rating evaluation model for rating the energy-efficiency level of android google play apps. *International Journal of Electrical and Computer Engineering* 11, 2 (2021).
- [9] Tiago L Alves, Christiaan Ypma, and Joost Visser. 2010. Deriving metric thresholds from benchmark data. In *2010 IEEE international conference on software maintenance*. IEEE, 1–10.
- [10] Anon. 2025. GitHub - tmp351/BRACE: Unified Benchmarking Accuracy and Energy of Code Language Models – github.com. <https://github.com/tmp351/BRACE>. [Accessed 13-10-2025].
- [11] Mauricio Fadel Argerich and Marta Patiño-Martínez. 2024. Measuring and improving the energy efficiency of large language models inference. *IEEE Access* 12 (2024).
- [12] Arne Berger, Matthias Knauer, and Christof Büskens. 2018. Pareto Front Interpolation Based on Parametric Sensitivity Analysis in a Bi-Objective Setting. *PAMM* 18, 1 (2018).
- [13] Nuran Bradley. 2007. *The response surface methodology*. Ph.D. Dissertation. Indiana University South Bend.
- [14] Jonathan Bright, Florence Enock, Saba Esnaashari, John Francis, Youmna Hashem, and Deborah Morgan. 2025. Generative AI is already widespread in the public sector: evidence from a survey of UK public sector professionals. *Digit. Gov: Res. Pract.* 6, 1 (2025). doi:10.1145/3700140
- [15] Alexander EI Brownlee, Jason Adair, Saemundur O Haraldsson, and John Jabbo. 2021. Exploring the accuracy–energy trade-off in machine learning. In *2021 IEEE/ACM International Workshop on Genetic Improvement (GI)*. IEEE.
- [16] David Budesu. 1980. A Note On Polynomial Regression. *Multivariate Behavioral Research* 15 (1980). doi:10.1207/s15327906mbr1504_7
- [17] Ethan Caballero, Kshitij Gupta, Irina Rish, and David Krueger. 2023. Broken Neural Scaling Laws. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=sckjveqlCZ>
- [18] Francisco Caravaca, Ángel Cuevas, and Rubén Cuevas. 2025. From Prompts to Power: Measuring the Energy Footprint of LLM Inference. *arXiv preprint arXiv:2511.05597* (2025).
- [19] Catherine Cazals, Jean-Pierre Florens, and Léopold Simar. 2002. Nonparametric frontier estimation: a robust approach. *Journal of econometrics* 106, 1 (2002).
- [20] Aydın Çelen. 2014. Comparative analysis of normalization procedures in TOPSIS method: with an application to Turkish deposit banking market. *Informatica* (2014).
- [21] A. Charnes, W.W. Cooper, and E. Rhodes. 1978. Measuring the efficiency of decision making units. *European Journal of Operational Research* 2, 6 (1978). doi:10.1016/0377-2217(78)90138-8
- [22] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [23] David Christensen and Kirk Payne. 1992. Cost performance index stability: fact or fiction? *Journal of Parametrics* (1992).
- [24] Jae-Won Chung, Jiachen Liu, Jeff J. Ma, Ruofan Wu, Oh Jun Kweon, Yuxuan Xia, Zhiyu Wu, and Mosharaf Chowdhury. 2025. The MLENERGY Benchmark: Toward Automated Inference Energy Measurement and Optimization. *arXiv preprint arXiv:2505.06371* (2025).
- [25] Charles W Cobb and Paul H Douglas. 1928. A theory of production. *The American economic review* (1928).
- [26] R. De Maesschalck, D. Jouan-Rimbaud, and D.L. Massart. 2000. The Mahalanobis distance. *Chemometrics and Intelligent Laboratory Systems* 50, 1 (2000). doi:10.1016/S0169-7439(99)00047-7
- [27] Zhengxiao Du, Aohan Zeng, Yuxiao Dong, and Jie Tang. 2024. Understanding emergent abilities of language models from the loss perspective. *Advances in neural information processing systems* 37 (2024), 53138–53167.
- [28] Hugging Face. 2025. Hugging Face. <https://huggingface.co/>. Accessed: 2025-10-10.
- [29] Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Chukwunyere Osi, Prateek Sharma, Fan Chen, and Lei Jiang. 2024. LLMCarbon: Modeling the End-to-End Carbon Footprint of Large Language Models. In *The Twelfth International Conference on Learning Representations*.
- [30] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. doi:10.1109/ICSE-FoSE59343.2023.00008
- [31] Emile Dos Santos Ferreira, Andrei Paleyes, and Neil D Lawrence. 2026. Optimising for Energy Efficiency and Performance in Machine Learning. In *2026 IEEE/ACM 5th International Conference on AI Engineering–Software Engineering for AI (CAIN)*. IEEE.
- [32] Raphael Fischer, Matthias Jakobs, Sascha Mücke, and Katharina Morik. 2022. A unified framework for assessing energy efficiency of machine learning. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer.

- [33] Clémentine Fourrier, Nathan Habib, Alina Lozovskaya, Konrad Szafer, and Thomas Wolf. 2024. Open LLM Leaderboard v2.
- [34] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2024. The Language Model Evaluation Harness. [doi:10.5281/zenodo.12608602](https://doi.org/10.5281/zenodo.12608602)
- [35] Chunmian Ge and Ke-Wei Huang. 2014. Analyzing the economies of scale of software as a service software firms: A stochastic frontier approach. *IEEE Transactions on Engineering Management* 61, 4 (2014).
- [36] GenAI Impact. 2025. EcoLogits Documentation. <https://ecologits.ai/latest/>. Accessed 2025-09-09.
- [37] GitHub. 2025. *GitHub Copilot*. GitHub, Inc. <https://github.com/features/copilot>
- [38] Ana Gjorgjevikj, Ana Nikolikj, Barbara Koroušić Seljak, and Tome Eftimov. 2025. User-defined trade-offs in LLM benchmarking: balancing accuracy, scale, and sustainability. *Knowledge-Based Systems* (2025).
- [39] Yash Goel, Ayan Sengupta, and Tanmoy Chakraborty. 2025. Position: Enough of Scaling LLMs! Lets Focus on Downscaling. *arXiv preprint arXiv:2505.00985* (2025).
- [40] Maria D Guillen, Juan Aparicio, José L Zofio, and Victor J Espana. 2024. Improving the predictive accuracy of production frontier models for efficiency measurement using machine learning: The LSB-MAFS method. *Computers & Operations Research* 171 (2024).
- [41] Janis Hardwick and Quentin F Stout. 2014. Optimal reduced isotonic regression. *arXiv preprint arXiv:1412.2844* (2014).
- [42] S Haseena, M Blessa Binolin Pepsi, and S Saroja. 2020. Multi criteria decision making technique for machine learning algorithms: iterative and noniterative algorithms. *International Journal Of Scientific & Technology Research* 9, 1 (2020), 2392–2403.
- [43] Riko Hendrawan, Kristian Nugroho, and Gayuh Permana. 2020. Efficiency Perspective on Telecom Mobile Data Traffic. *GATR Journal of Business and Economics Review* 5 (03 2020), 38–44. [doi:10.35609/jber.2020.5.1\(5\)](https://doi.org/10.35609/jber.2020.5.1(5))
- [44] Danny Hernandez, Jared Kaplan, Tom Henighan, and Sam McCandlish. 2021. Scaling laws for transfer. *arXiv preprint arXiv:2102.01293* (2021).
- [45] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8 (2024). [doi:10.1145/3695988](https://doi.org/10.1145/3695988)
- [46] Mia Hubert and Michiel Debruyne. 2010. Minimum covariance determinant. *Wiley interdisciplinary reviews: Computational statistics* 2, 1 (2010).
- [47] Erik Johannes Husom, Arda Goknil, Merve Astekin, Lwin Khin Shar, Andre Käsen, Sagar Sen, Benedikt Andreas Mithassel, and Ahmet Soylu. 2025. Sustainable LLM inference for edge AI: Evaluating quantized llms for energy efficiency, output accuracy, and inference latency. *ACM Transactions on Internet of Things* (2025).
- [48] Ching-Lai Hwang and Kwangsun Yoon. 1981. *Methods for Multiple Attribute Decision Making*. Springer Berlin Heidelberg. [doi:10.1007/978-3-642-48318-9_3](https://doi.org/10.1007/978-3-642-48318-9_3)
- [49] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations*.
- [50] Nidhal Jegham, Marwan Abdelatti, Lassad Elmoubarki, and Abdeltawab Hendawi. 2025. How hungry is AI? benchmarking energy, water, and carbon footprint of LLM inference. *arXiv preprint arXiv:2505.09598* (2025).
- [51] Hamed Jelodar, Mohammad Meymani, and Roozbeh Razavi-Far. 2025. Large language models (LLMs) for source code analysis: applications, models and datasets. *arXiv preprint arXiv:2503.17502* (2025).
- [52] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2025. A Survey on Large Language Models for Code Generation. *ACM Trans. Softw. Eng. Methodol.* (2025). [doi:10.1145/3747588](https://doi.org/10.1145/3747588)
- [53] Yunho Jin, Gu-Yeon Wei, and David Brooks. 2025. The Energy Cost of Reasoning: Analyzing Energy Usage in LLMs with Test-time Compute. *arXiv preprint arXiv:2505.14733* (2025).
- [54] Angelika Kaplan, Jan Keim, Lukas Greiner, Ralf Sieger, Raffaella Mirandola, and Ralf Reussner. 2025. Responsible and Sustainable AI: Considering Energy Consumption in Automated Text Classification Evaluation Tasks. In *Proceedings of the 2025 IEEE/ACM 9th International Workshop on Green and Sustainable Software (GREENS '25)*. IEEE Press. [doi:10.1109/GREENS66463.2025.00016](https://doi.org/10.1109/GREENS66463.2025.00016)
- [55] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [56] Jakub Krajewski, Amitis Shidani, Dan Busbridge, Sam Wiseman, and Jason Ramapuram. 2026. Revisiting the Scaling Properties of Downstream Metrics in Large Language Model Training. *The Fourteenth International Conference on Learning Representations* (2026). <https://openreview.net/forum?id=YnJ2s4WeNF>
- [57] Arindam Kundu, Sumit Kumar, Nutan Kumar Tomar, and Shiv Kumar Gupta. 2016. Call option price function in Bernstein polynomial basis with no-arbitrage inequality constraints. *Journal of Inequalities and Applications* 2016, 1 (2016).
- [58] Pengfei Li, Jianyi Yang, Mohammad A Islam, and Shaolei Ren. 2025. Making AI less 'thirsty'. *Commun. ACM* 68, 7 (2025).
- [59] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. 2022. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110* (2022).
- [60] Chin-Yew Lin and Franz Josef Och. 2004. Orange: a method for evaluating automatic evaluation metrics for machine translation. In *COLING 2004: Proceedings of the 20th International Conference on Computational Linguistics*.
- [61] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, MING GONG, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and

- Shujie LIU. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*.
- [62] Alexandra Sasha Luccioni, Sylvain Viguier, and Anne-Laure Ligozat. 2023. Estimating the carbon footprint of bloom, a 176b parameter language model. *Journal of machine learning research* 24 (2023).
- [63] Sasha Luccioni, Boris Gamazaychikov, Emma Strubell, Sara Hooker, Yacine Jernite, Carole-Jean Wu, and Margaret Mitchell. 2025. AI Energy Score Leaderboard - February 2025.
- [64] Sasha Luccioni, Yacine Jernite, and Emma Strubell. 2024. Power hungry processing: Watts driving the cost of AI deployment?. In *Proceedings of the 2024 ACM conference on fairness, accountability, and transparency*.
- [65] B Mahaboob, KA Ajmath, B Venkateswarlu, C Narayana, and J Peter Praveen. 2019. On Cobb-Douglas production function model. In *AIP Conference Proceedings*. AIP Publishing LLC.
- [66] Felipe Maia Polo, Seamus Somerstep, Leshem Choshen, Yuekai Sun, and Mikhail Yurochkin. 2026. Sloth: scaling laws for llm skills to predict multi-benchmark performance across families. *Advances in Neural Information Processing Systems* (2026).
- [67] Miguel Angel Quiroz Martinez, Eduardo Xavier Moreira Bermello, Wilmer Paul Carrillo Leon, and Luis Briones. 2022. A Hybrid Approach Entropy-TOPSIS for the Selection of Machine Learning Classifiers for Software Defect Prediction. *Intelligent Human Systems Integration (IHSI 2022): Integrating People and Intelligent Systems* 22, 22 (2022).
- [68] Elio Masciari, Vincenzo Moscato, Enea Vincenzo Napolitano, Gian Marco Orlando, Marco Perillo, and Diego Russo. 2026. Are LLMs Ready for TOON? Benchmarking Structural Correctness-Sustainability Trade-offs in Novel Structured Output Formats. *arXiv preprint arXiv:2601.12014* (2026).
- [69] Antonio Mastropaolo, Camilo Escobar-Velásquez, and Mario Linares-Vásquez. 2025. From triumph to uncertainty: The journey of software engineering in the AI era. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025).
- [70] Manoj Mathew, Sagar Sahu, and Ashish Kumar Upadhyay. 2017. Effect of normalization techniques in robot selection using weighted aggregated sum product assessment. *Int. J. Innov. Res. Adv. Stud* (2017).
- [71] Joseph McDonald, Baolin Li, Nathan Frey, Devesh Tiwari, Vijay Gadepally, and Siddharth Samsi. 2022. Great Power, Great Responsibility: Recommendations for Reducing Energy for Training Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2022*. Association for Computational Linguistics. doi:10.18653/v1/2022.findings-naacl.151
- [72] Ali Mohaghar. 2013. Developing TOPSIS method using statistical normalization for selecting knowledge management strategies. *Journal of industrial engineering and management* (2013).
- [73] J Morrison, C Na, J Fernandez, T Dettmers, E Strubell, and J Dodge. 2025. Holistically Evaluating the Environmental Impact of Creating Language Models. *arXiv 2025. arXiv preprint arXiv:2503.05804* (2025).
- [74] Sophia Nguyen, Beihao Zhou, Yi Ding, and Sihang Liu. 2024. Towards sustainable large language model serving. *ACM SIGENERGY Energy Informatics Review* 4, 5 (2024).
- [75] Chenxu Niu, Wei Zhang, Yongjian Zhao, and Yong Chen. 2025. Energy efficient or exhaustive? benchmarking power consumption of llm inference engines. *ACM SIGENERGY Energy Informatics Review* (2025).
- [76] Michael Nordine, Anton Schwarz, Renana Bruckstein, Hanns-Christian Gunga, and Oliver Opatz. 2022. The human dive reflex during consecutive apnoeas in dry and immersive environments: magnitude and synchronicity. *Frontiers in Physiology* 12 (2022), 725361.
- [77] Omer Onalan and Hulya Basegmez. 2018. Estimation of economic growth using Grey Cobb-Douglas production function: An application for US economy. *Journal of Business Economics and Finance* (2018).
- [78] Zuzanna Osika, Jazmin Zatarain Salazar, Diederik M. Roijers, Frans A. Oliehoek, and Pradeep K. Murukannaiah. 2023. What lies beyond the pareto front? a survey on decision-support methods for multi-objective optimization. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI '23)*. doi:10.24963/ijcai.2023/755
- [79] David Owen. 2024. How predictable is language model benchmark performance? *arXiv preprint arXiv:2401.04757* (2024).
- [80] Nghiem Thanh Pham, Tung Kieu, Duc-Manh Nguyen, Son Ha Xuan, Nghia Duong-Trung, and Danh Le-Phuoc. 2025. SLM-Bench: A Comprehensive Benchmark of Small Language Models on Environmental Impacts-Extended Version. *arXiv preprint arXiv:2508.15478* (2025).
- [81] Ketai Qiu, Niccolò Puccinelli, Matteo Ciniselli, and Luca Di Grazia. 2025. From Today's Code to Tomorrow's Symphony: The AI Transformation of Developer's Routine by 2030. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (2025). doi:10.1145/3709353
- [82] Saurabhsingh Rajput, Mootez Saad, and Tushar Sharma. 2025. Tu(r)ning AI Green: Exploring Energy Efficiency Cascading with Orthogonal Optimizations. *arXiv preprint arXiv:2506.18289* (2025).
- [83] Saurabhsingh Rajput and Tushar Sharma. 2024. Benchmarking Emerging Deep Learning Quantization Methods for Energy Efficiency. In *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*. doi:10.1109/ICSA-C63560.2024.00049
- [84] Saurabhsingh Rajput and Tushar Sharma. 2026. CodeGreen: Towards Improving Precision and Portability in Software Energy Measurement. *Proceedings of the ACM on Software Engineering FSE* (2026). https://arxiv.org/abs/2603.17924
- [85] Saurabhsingh Rajput, Tim Widmayer, Ziyuan Shang, Maria Kechagia, Federica Sarro, and Tushar Sharma. 2024. Enhancing energy-awareness in deep learning through fine-grained energy measurement. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024).
- [86] Peter Rousseeuw. 1985. Multivariate estimation with high breakdown point. *Mathematical statistics and applications* 8 (1985).
- [87] Peter J Rousseeuw. 1984. Least median of squares regression. *Journal of the American statistical association* 79 (1984).

- [88] Yangjun Ruan, Chris J Maddison, and Tatsunori Hashimoto. 2024. Observational scaling laws and the predictability of language model performance. *Advances in Neural Information Processing Systems* (2024).
- [89] Mootez Saad, José Antonio Hernández López, Boqi Chen, Daniel Varró, and Tushar Sharma. 2025. An Adaptive Language-Agnostic Pruning Method for Greener Language Models for Code. *Proceedings of the ACM on Software Engineering* 2, FSE (2025).
- [90] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, and Vijay Gadepally. 2023. From words to watts: Benchmarking the energy costs of large language model inference. In *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE.
- [91] Scikit-Learn. 2025. Isotonic Regression. <https://scikit-learn.org/stable/modules/isotonic.html>. Accessed 2025-09-24.
- [92] Won-Gyo Seo and Se-Jin Han. 2017. Comparison of the effects on the pharyngeal airway space of maxillary protraction appliances according to the methods of anchorage. *Maxillofacial plastic and reconstructive surgery* 39, 1 (2017), 3.
- [93] Zhigang Shen, Ashkan Hassani, and Qian Shi. 2016. Multi-objective time-cost optimization using Cobb-Douglas production function and hybrid genetic algorithm. *Journal of Civil Engineering and Management* (2016).
- [94] Olivier Sobrie, Nicolas Gillis, Vincent Mousseau, and Marc Pirlot. 2018. UTA-poly and UTA-splines: additive value functions with polynomial marginals. *European Journal of Operational Research* 264, 2 (2018).
- [95] Ankita Nandkishor Sontakke, Manasi Patwardhan, Lovekesh Vig, Raveendra Kumar Medicherla, Ravindra Naik, and Gautam Shroff. 2022. Code summarization: Do transformers really understand code?. In *Deep Learning for Code Workshop*.
- [96] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adri Garriga-Alonso, et al. 2023. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on machine learning research* (2023).
- [97] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2020. Energy and policy considerations for modern deep learning research. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34.
- [98] Gemma Team, T Mesnard, C Hardin, R Dadashi, S Bhupatiraju, S Pathak, L Sifre, M Rivière, MS Kale, J Love, et al. [n. d.]. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295* ([n. d.]).
- [99] Kaoru Tone. 2001. A slacks-based measure of efficiency in data envelopment analysis. *European journal of operational research* 130, 3 (2001).
- [100] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [101] Gwo-Hshiung Tzeng and Jih-Jeng Huang. 2011. *Multiple attribute decision making: methods and applications*. CRC press.
- [102] Ruiqi Wang, Jiyu Guo, Cuiyun Gao, Guodong Fan, Chun Yong Chong, and Xin Xia. 2025. Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering. *Proc. ACM Softw. Eng.* 2, ISSTA (2025). [doi:10.1145/3728963](https://doi.org/10.1145/3728963)
- [103] Jinfeng Wen, Zhenpeng Chen, Jianshu Zhao, Federica Sarro, Haodi Ping, Ying Zhang, Shangguang Wang, and Xuanzhe Liu. 2025. SCOPE: Performance testing for serverless computing. *ACM Transactions on Software Engineering and Methodology* (2025).
- [104] Chaojun Xiao, Jie Cai, Weilin Zhao, Biyuan Lin, Guoyang Zeng, Jie Zhou, Zhi Zheng, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. Densing law of llms. *Nature Machine Intelligence* (2025).
- [105] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
- [106] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. 2023. Zeus: Understanding and Optimizing GPU Energy Consumption of DNN Training. In *USENIX NSDI*.
- [107] Matt Young. 2006. The stone-weierstrass theorem. In *MATH 328 Notes*. Queen's University at Kingston.
- [108] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging LLM-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023).
- [109] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-Following Evaluation for Large Language Models. *arXiv:2311.07911* [cs.CL]
- [110] Stanley Zions. 1979. MCDM—If not a roman numeral, then what? *Interfaces* 9, 4 (1979).
- [111] Johannes Zschache and Tilman Hartwig. 2025. Comparing energy consumption and accuracy in text classification inference. *arXiv preprint arXiv:2508.14170* (2025).
- [112] Fida Zubair, Maryam Al-Hitmi, and Cagatay Catal. 2025. The use of large language models for program repair. *Computer Standards & Interfaces* 93 (2025). [doi:10.1016/j.csi.2024.103951](https://doi.org/10.1016/j.csi.2024.103951)